RMIT Classification: Trusted

RMIT University

Bachelor of Engineering (Robotics and Mechatronics Engineering) (Honours) BH123

OENG1185/COSC2503 Engineering & IT Capstone Project

Capstone Project Final Report

3-DOF Stewart Platform with Hardware-in-the-Loop Verification using the Bosch Rexroth ctrlX Ecosystem

Student team:	The Boys
Student name & ID:	Pham Tri Dung s3981129
	Le Huu Toan s3979542
	Nguyen Vu Tran Khoi s3978262
	Pham Quan s4045107
	Wong Vi Khang s3978666
Academic supervisor:	Dr. Hai Nguyen Nguyen
Company:	Bosch Rexroth
Industry supervisor:	Mr. Nguyen Hoang Thinh
	Mr. Nguyen Hung Binh
	Mr. Krasilnikov Aleksandr

Contents

1 Executive summary 6
2 Project background & problem statement 7
2.1 Project background 7
2.2 Problem statement 7
2.3 Scope of impact 7
3 Literature review & market research 8
3.1 Application 8
3.2 Market realisation 8
3.2.1 Design of stabilisation platforms 8
3.2.2 Verification methods 10
3.3 Hardware choice 11
3.3.1 Actuator 11
3.3.2 Sensor 11
3.3.3 Controller 12
4 Solution design 13
4.1 Scope of work and performance metrics 13
4.1.1 Scope of work 13
4.1.2 Performance metrics 13
4.1.3 Design alternatives and selection 14
4.2 System architecture 15
4.2.1 Electrical and control architecture 15
4.2.2 Mechanical architecture 15
4.3 Stewart kinematics 16
4.4 Digital twin and verification strategy 17
4.4.1 Simscape digital twin 17
4.4.2 Verification method 19
4.5 Embedded implementation 21
4.5.1 Embedding code into the STM32 21
4.6 Integration with Bosch 23
4.6.1 Bosch PLC implementation 23
4.6.2 Monitoring using HMI 25
4.6.3 3D Viewer digital twin 27
4.7 Prototype design 28
4.7.1 Arduino working prototype 28
4.7.2 Working prototype Stewart platform 28
4.8 Industrial design 29

5 Result analyses & discussions 30
5.1 Result analysis 30
5.1.1 Digital twin versus physical platform 30
5.2 Performance evaluation 30
5.3 Discussion 31
5.3.1 Achievement of objectives 31
5.3.2 Project timeline 31
5.3.3 Risk assessment 32
5.3.4 Limitations and future work 33
6 Summary and reflection 34
7 References 35
8 Appendices 37
8.1 Bill of materials 37
8.2 Electrical drawings 38
8.3 Mechanical drawings 39
8.4 Control code 44
8.5 Project timeline 44
9 Meeting journals 45

List of Figures

Figure 3-1: Industrial applications of the Stewart platform [4]–[7] 8
Figure 3-2: Target application: active orientation rig for drone wind tunnel testing [8] 8
Figure 3-3: Single-axis stabilisation platform [9] 9
Figure 3-4: Multi-axis gimbal 9
Figure 3-5: Parallel manipulator (Stewart platform) [10] 9
Figure 3-6: Model-in-the-Loop (MIL) 10
Figure 3-7: Speedgoat real-time target machine for HIL testing [11] 11
Figure 3-8: Stepper motor electric cylinder [12] 11
Figure 3-9: STM32 NUCLEO-H753ZI development board [13] 12
Figure 4-1: Electrical and control architecture 15
Figure 4-2: Mechanical architecture of the 3-RPS platform [14] 15
Figure 4-3: Closed-loop vector geometry of the 3-RPS platform [15] 16
Figure 4-4: Simscape Multibody model of the Stewart platform 18
Figure 4-5: Simscape link, joint, and configurable joint parameters 18
Figure 4-6: Serial Configuration and Serial Receive block parameters 19
Figure 4-7: MIL test setup 20
Figure 4-8: Parallel test of the physical platform and digital twin 20
Figure 4-9: STM32MX Pins configuration 21
Figure 4-10: Modular organisation of the control algorithm 21
Figure 4-11: Homing procedure for the three linear actuators 22
Figure 4-12: Inverse Kinematics Function Block in ctrlX PLC Engineering 23
Figure 4-13: Variables of the Inverse Kinematics Function Block 24
Figure 4-14: Implementation of the inverse kinematics in the ctrlX CORE 24
Figure 4-15: Variables used for translational motion 25
Figure 4-16: Inverse Kinematics Function Block in the Ladder Logic main program 25
Figure 4-17: Ladder Logic control of axis 1 25
Figure 4-18: Web-based HMI for the Stewart platform 26
Figure 4-19: Stewart platform digital twin in the Bosch Rexroth 3D Viewer 28
Figure 4-20: Rack-and-pinion travel range of the prototype actuator 28
Figure 8-1: Electrical wiring diagram of the control panel 38
Figure 8-2: Stewart Platform Isometric view 41
Figure 8-3: Electrical cylinder 41
Figure 8-4: KP08 Pillow Bearing 42
Figure 8-5: 3D printed Cover Top 42
Figure 8-6: Base 42
Figure 8-7: 3D printed ball joint 43
Figure 8-8: 3D printed Top attachment 43
Figure 8-9: Revolute joint pin 43
Figure 8-10: Project Gantt chart (Completion Plan schedule) 44

List of Tables

Table 4-1: Performance metrics for the final system 13
Table 4-2: Selection of the platform architecture 14
Table 4-3: Selection of the actuator for the industrial build 14
Table 4-4: Selection of the main controller 14
Table 4-5 Components of the working prototype 29
Table 5-1: Comparison between commanded and measured platform poses 30
Table 5-2: Evaluation against the performance metrics 30
Table 5-3: Deliverables versus outcomes 31
Table 5-4: Risk mitigation action matrix 32
Table 8-1: Control panel components 38
Table 8-2: Mechanical components of the industrial Stewart platform 40

1 Executive summary

Industrial parallel robotic platforms are mechanically mature, but a persistent gap remains between high-level simulation and physical control: control algorithms are often validated directly on live hardware, risking mechanical damage, cost overruns, and delays, while a lack of standardised, bi-directional interfaces between simulation environments and industrial controllers keeps development cycles slow and hardware dependent.

This project involves designing and building a 3-DOF Stewart Platform with a model-based verification pipeline aimed at Hardware-in-the-Loop (HIL), in partnership with the Bosch Rexroth ecosystem. To close the simulation–reality gap, the team solves the inverse kinematics onboard an STM32H7 microcontroller and validates the control logic through a staged Model-in-the-Loop / Processor-in-the-Loop progression before deployment to the physical actuators, keeping the digital twin driven by the same values sent to the hardware. Furthermore, the team has developed a working industrial system using an IMU, an STM32H7, and stepper motor linear actuators, alongside a web app dashboard and 3D digital twin on ctrlX CORE Virtual.

The target use case of this project is an active orientation rig for drone wind tunnel testing. Beyond this, the robot's applications vary widely, from balancing objects and stabilising camera or sensor payloads, to serving as a motion base for flight and driving simulators, to precision alignment tasks such as telescope pointing and optical bench positioning — any use case that benefits from controlled heave, roll, and pitch motion.

The user requirement is that the user enters a final pose, and the robot moves to that pose by changing the linear actuator lengths. The system supports two control modes: manual control via PuTTY, and IMU-based control for a more interactive experience. Since the physical ctrlX CORE PLC was not provided, the Bosch Rexroth side runs on ctrlX CORE Virtual, which hosts the web app HMI, the inverse kinematics Function Block, and the 3D Viewer digital twin, operating independently of the STM32.

The design process consists of a user requirement definition phase (aligning scope and objectives with all stakeholders), initial development, mid- and final-stage development, integration with Bosch Rexroth, building a working prototype, and building the industrial working robot.

The results of the project include a complete working prototype, a demonstrable industrial robot with interactive telemetry, the full control code, and a Bosch Rexroth digital twin and HMI on ctrlX CORE Virtual. Further development targets connecting the STM32 to the ctrlX CORE, finalising the web app/HMI hosting platform, completing full end-to-end safety validation, and extending verification toward broader Hardware-in-the-Loop coverage as the system moves toward final industrial deployment.

2 Project background & problem statement

2.1 Project background

While 3-DOF parallel platform technology is mature, it often lacks seamless integration between MATLAB/Simulink models and modern cloud-based platforms such as the Bosch ctrlX ecosystem, frequently relying on closed systems. Our project originally aimed to address this by modelling the platform's kinematics in Simulink, tuning the control loops, generating C code for the control algorithm, and integrating it with ctrlX motion hardware via Hardware-in-the-Loop (HIL) simulation. As the physical ctrlX CORE was not provided, the verification scope was revised to MIL and PIL with an STM32 controller (Section 4.1.1). We have implemented the embedded control layer on an STM32H7 (state machine, stepper driver interface, onboard inverse kinematics) and validated the Simulink/Simscape digital twin pipeline over a serial link, sending computed leg-length outputs directly to the plant model to preserve HIL fidelity. In parallel, our Bosch-integration sub-team has developed the ctrlX CORE Virtual side — the inverse kinematics Function Block, the OPC UA-based web HMI and the 3D Viewer digital twin; linking it to the STM32 remains future work.

Furthermore, traditional platforms rely on local consoles and dedicated units, whereas industry is shifting toward Software-Defined Automation. In recent years, cloud computing has gained huge market success worldwide by offering flexible and scalable computing capability and storage space [1]. To meet Industry 4.0 requirements for remote, web-based operation, our project targets a .SNAP web server dashboard that moves simulation and control of the 3-RPS parallel platform to a cloud-ready environment hosted on the ctrlX CORE device.

2.2 Problem statement

Despite the mechanical maturity of parallel robotic structures, current industrial workflows face a critical simulation–reality gap. This bottleneck exists because traditional control methods lack a standardised, bi-directional interface between high-level MATLAB/Simulink models and modern industrial hardware such as the ctrlX CORE [2]. Consequently, engineers are often forced into a high-risk development cycle where unverified control algorithms are tested directly on physical hardware — increasing the risk of mechanical failure, financial loss, and preventable delays.

To ensure baseline control simplicity and real-time computation stability, we solve the inverse kinematics onboard the STM32 microcontroller rather than in the higher-level control loop, avoiding intensive dynamic force calculations while keeping the kinematics and motor-control layers decoupled through a thin software interface. This lets the digital twin be driven by the same leg-length values the physical drivers receive, rather than a separately recomputed model — preserving HIL fidelity.

Furthermore, the absence of Software-Defined remote access limits the capability to monitor or tune these control loops safely [3], forcing a rigid dependency on on-site operation that is increasingly incompatible with modern safety requirements and the SDG 2030 goals for sustainable, resilient industrial infrastructure.

2.3 Scope of impact

Our project enhances the integration between Stewart platform simulation and control via Hardware-in-the-Loop (HIL) using the ctrlX ecosystem – the cutting-edge industrial platform from Bosch Rexroth. By utilising HIL, we preserve hardware integrity and ensure operator safety while significantly reducing testing costs. Once the simulation runs within the standardised ctrlX environment, the company will benefit from a unified development workflow, further accelerating time-to-market.

Additionally, the development of a web-based application will enable engineers and operators to work remotely from the safety of an office, effectively removing them from the hazards and noise of the factory floor. This shift fosters a safer, more ergonomic working environment. By participating in this initiative, students bridge the gap between academic research and practical engineering. This dual exposure strengthens technical competence, industry readiness, and career orientation within the realm of the Fourth Industrial Revolution. Finally, this project aligns with the national commitment to SDG 2030. It supports SDG 3 (Good Health and Well-being) by improving occupational safety and reducing exposure to hazardous industrial environments (Target 3.9). Furthermore, it advances SDG 9 (Industry, Innovation and Infrastructure) by upgrading industrial infrastructure for sustainability (Target 9.4) and enhancing scientific research through advanced automation technologies (Target 9.5).

3 Literature review & market research

3.1 Application

The main focus and stakeholder requirement for our platform design is to stabilise objects and control end-effector orientation. Stabilisation and orientation platforms neutralise external motion and vibration to keep payloads, cameras, or systems level and accurately pointed.

In industry, these platforms are used in many applications, ranging from marine and offshore (stabilisation platforms for offshore oil rigs, sensor mounts) and aerospace and drones (flight simulators, international docking systems) to orientation platforms for telescopes.

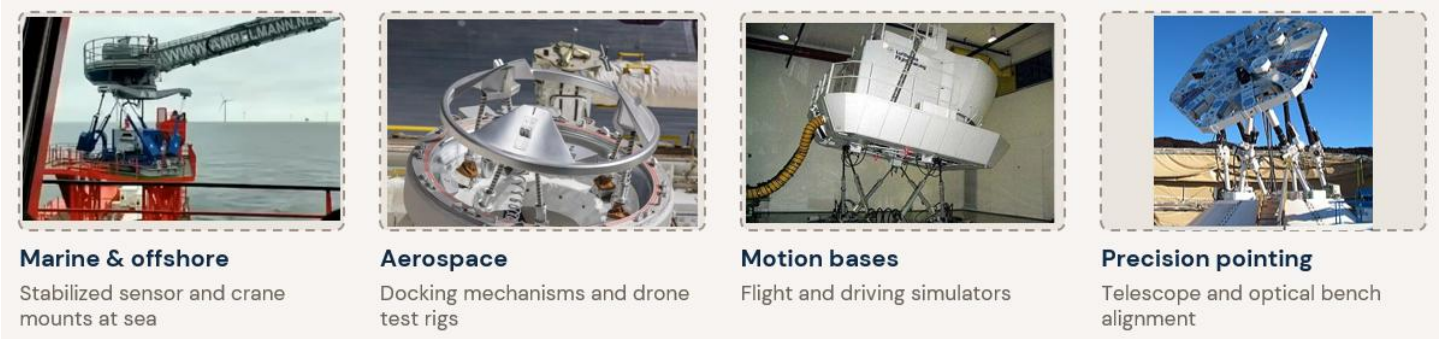


Figure 3-1: Industrial applications of the Stewart platform [4]–[7]

Specifically, our product is intended as an active orientation testing rig for drones in wind tunnel simulation tests, where the platform, attached to the drone, moves actively to simulate the behaviour of the drone in flight.

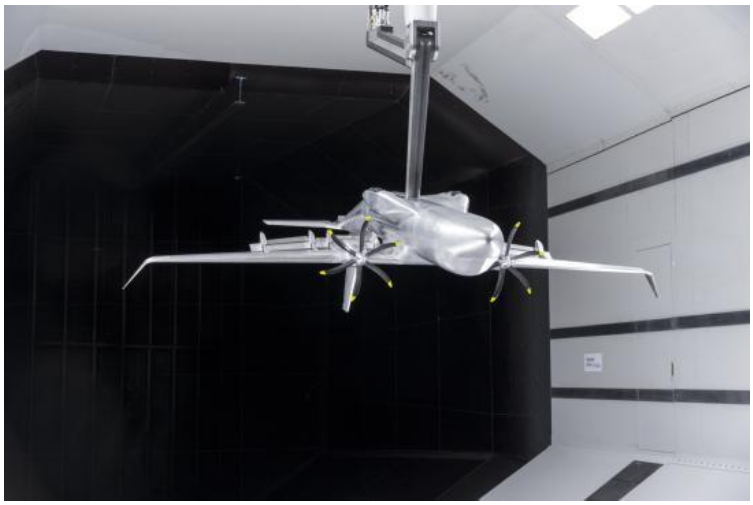


Figure 3-2: Target application: active orientation rig for drone wind tunnel testing [8]

3.2 Market realisation

3.2.1 Design of stabilisation platforms

Major types by architecture:

- Single-axis platforms: Rotate around one axis of movement (such as azimuth or elevation) to stabilise or point payloads such as flat-panel antennas on the move.



Figure 3-3: Single-axis stabilisation platform [9]

- Multi-axis (2-axis and 3-axis) gimbals: Use a nested frame structure providing rotational freedom across pitch, roll, and yaw to completely isolate the payload from vehicle or vessel rotation.



Figure 3-4: Multi-axis gimbal

- Parallel manipulators (e.g., Stewart platforms / 3-RPS): Use multiple coordinated linear actuators connected to a single top plate to control up to six degrees of freedom (3 linear axes and 3 rotational axes) for heavy payloads.



Figure 3-5: Parallel manipulator (Stewart platform) [10]

The requirement for this project is a platform that allows movement in pitch, roll, and heave. It also requires good reaction speed, good payload capacity, and precise movement.

The Stewart platform offers unmatched structural rigidity, extreme precision, and full three-degree-of-freedom (3-DoF) motion in a highly compact design; therefore, the Stewart platform is suitable for both the movement and payload requirements.

Regarding the comparison between 3-DOF and 6-DOF platforms, the 3-DOF platform matches the requirement from Bosch, is about 2× cheaper (3 vs 6 cylinders), and is also much better in terms of kinematic simplicity.

Commercial 3-DOF platforms in the hobbyist/education space typically use RC servos or small linear actuators and cost from a few hundred to a few thousand USD, while industrial-grade platforms with closed-loop stepper/servo actuation and PLC-based control (comparable in spirit to the Bosch Rexroth ctrlX ecosystem originally scoped for this project) are used in manufacturing test rigs and simulators, and cost significantly more.

3.2.2 Verification methods

There are many testing protocols; however, the process below is the main and most important one for model-based design. Engineers use a multi-stage testing "funnel" to prove that systems work before final field deployment. This setup minimises risk (breaking equipment, assemblies not matching, etc.) and saves development time, cost, and resources.

MIL (Model-in-the-Loop): Tests pure mathematical and logical control algorithms inside a simulation software environment (such as MATLAB/Simulink) before writing any target code.

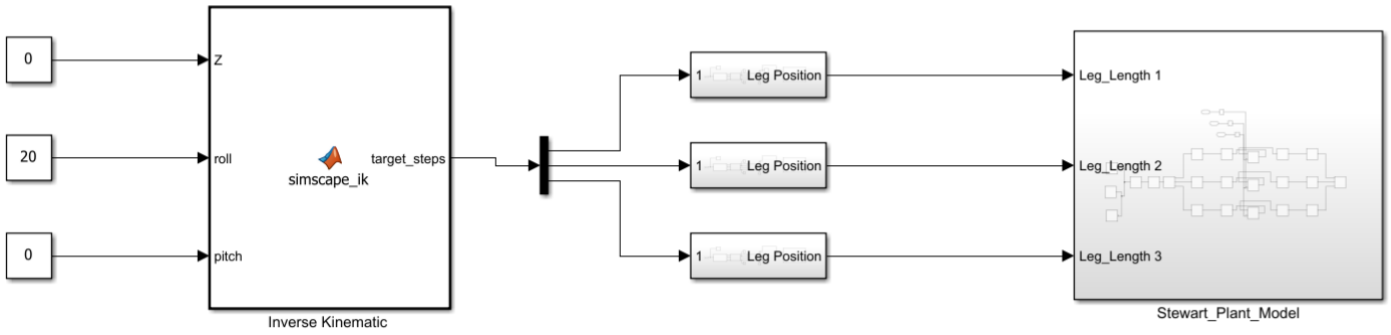


Figure 3-6: Model-in-the-Loop (MIL)

SIL (Software-in-the-Loop): Compiles the actual control software code and runs it within a simulated environment on a standard PC, without any physical hardware or microcontrollers.

PIL (Processor-in-the-Loop): Runs the compiled software directly on the target physical microcontroller or processor chip, while the environment and plant remain purely simulated on a host computer.

HIL (Hardware-in-the-Loop):

- Uses a digital twin running on a real-time target machine (e.g., Speedgoat) to act as the "plant", sensors, and actuators.
- Connects real physical hardware (the controller) to this digital environment via real analogue and digital signals.
- Allows safe testing of edge cases, faults, and failures without risking real-world equipment.



Figure 3-7: Speedgoat real-time target machine for HIL testing [11]

Field test / prototype: The final validation phase on the actual physical machine, vehicle, or plant under strict, controlled real-world operational oversight.

3.3 Hardware choice

This project sits between these two tiers: it uses industrial-grade design thinking (control, structured safety systems, HMI/telemetry) but is implemented on affordable, widely available STM32 and stepper motor hardware rather than a full industrial PLC stack. This makes the outcome relevant both as a student R&D deliverable and as a low-cost reference design for future capstone or lab teams who do not have guaranteed access to industrial PLC hardware.

3.3.1 Actuator

With the Stewart platform architecture chosen, the next choice is the actuator. The small-scale prototype uses RC servos. Stepper and brushless DC motors were not chosen for the prototype because RC servos excel at precise positioning and have an internal control loop and internal gears. This makes them simple to control and integrate. However, they also come with drawbacks. While RC servos serve very well in rapid prototyping and hobby projects, they are very unreliable: out of the 6 servo motors that we bought, 2 broke. Reliability is crucial in an industrial setting because if one motor breaks down, the line stops, which costs thousands of dollars in only one or two hours of downtime, in addition to the cost and lead time of replacement, shipping, and customs clearance.

Therefore, to choose the right motor for an industrial setting, a set of evaluations needs to be conducted. First, the industrial servo motor is regarded as the best option for us because of its performance, precision, and ease of integration. However, industrial servo motors are extremely expensive (10 million VND each, 30 million VND for 3 motors), so this option was discarded. Stepper motors, however, bring the same precision and very good holding torque at a much cheaper price. Furthermore, the stepper motor has moderate to fairly good speed (45mm/s). The only consideration is that we need to design the control algorithm to control the stepper motor and its driver at a low level, specifically through the EN, DIR, and PUL pins for enable, direction, and number of steps. Since we have the ability to solve that problem and develop the design, the stepper motor is regarded as the final option.

Stepper motors, though, come with minor drawbacks. The ability to deliver precise movement with high holding torque comes at a cost, which is safety, or the torque control problem. When commanded to move to a position, it moves without knowing the external force or torque applied to it. This can lead to catastrophic failure, as the motor will keep moving to that position and never stop midway, possibly breaking itself during the movement. To mitigate this, the top platform joints are made of plastic, which deforms easily, and a testing rig is used to protect the people around it.

However, considering all factors, from performance (precision and speed) to cost and ease of fabrication and assembly, an electric cylinder using a stepper motor tick all the boxes and is the best option, as our machine will only be used for demonstration and verification, not for actual production work.



Figure 3-8: Stepper motor electric cylinder [12]

3.3.2 Sensor

First, we need to understand the need for a sensor: what it is used for and which one is suitable. The sensor is used as a measuring device to evaluate the performance of the Stewart platform, specifically the final pose of the end effector.

The electric cylinder has a built-in Hall encoder, which fits best for closed-loop control of each leg, maximising precision while also providing feedback to our computer for verification and evaluation of leg performance.

However, as the cylinders were delivered later than expected, our team decided to drop the development of the Hall encoder. Two other types of sensor were then considered to measure the performance of the end effector: an IMU for the rotational pose (pitch, roll), and a laser sensor (mounted at the base, pointing at the end effector) for heave, since the IMU cannot reliably track linear movement and also suffers from drift. Due to time constraints, neither was implemented as a measuring device. Instead, the final pose is measured manually, with heave measured by a ruler or caliper and roll and pitch by a phone inclinometer (Section 5). The BNO055 IMU is retained in the final build as an interactive user input rather than a measuring device.

3.3.3 Controller

Since we are doing the project with Bosch Rexroth and using Bosch equipment, the controller is the Bosch PLC called ctrlX CORE. This device handles almost everything, from the HMI, web app control, and communication using OPC UA, to controlling the company's existing hardware (ctrlX DRIVE and the company's EMC electric cylinders).

However, due to some problems, the company could not provide us with a physical ctrlX CORE, so we changed to its virtual counterpart – ctrlX CORE Virtual – which has limited access and fewer features, and does not allow us to develop further, especially Hardware-in-the-Loop testing and digital twin modelling. This goes against the requirements; thus, the team decided on an alternative solution for the controller.

From multiple meetings and discussions, the team agreed on a more holistic and versatile approach, which is not only suitable for C programming but also cheap and easy to implement at home for HIL testing: using a microcontroller unit.

Three solutions were suggested: the ST STM32, the Espressif ESP32, and the Arduino UNO. The STM32 is well known for low-level hardware control, high clock speed, and very good real-time execution with motors and actuators. The ESP32, however, is very good at IoT, managing peripherals, and ease of coding and integration. The Arduino UNO offers the easiest coding, debugging, and integration of the three, with high-level coding; however, its real-time execution speed is much slower and its integration with peripherals (sensors, Ethernet port, etc.) is highly limited.

Given the requirements of real-time execution speed for HIL testing and integration with Bosch Rexroth, the STM32 is the top option of the three, followed by the ESP32. However, the ESP32 was not chosen because the team decided that an STM32 Nucleo board is easier to use, as it has existed plug-in pins and an Ethernet port (RJ45), and was available at the time. Thus, the STM32 is chosen as the main controller for this project.

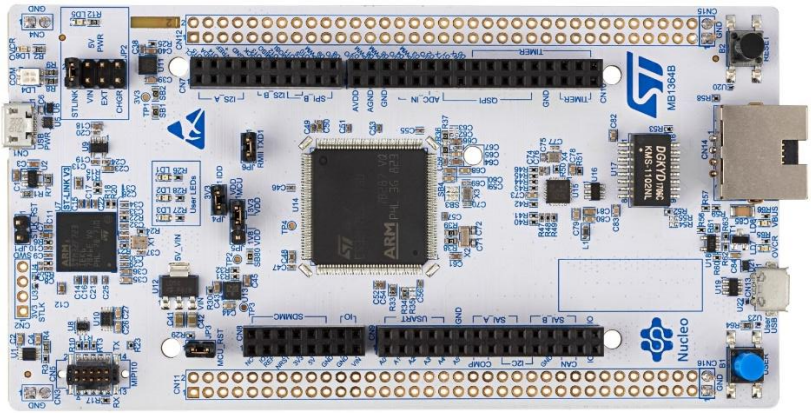


Figure 3-9: STM32 NUCLEO-H753ZI development board [13]

4 Solution design

4.1 Scope of work and performance metrics

4.1.1 Scope of work

Stakeholder requirements

Academic (SSET supervisor): Deliver a functioning R&D prototype demonstrating the mechanical platform and its control algorithm.

Industry (Bosch Rexroth): Requirements evolved as hardware availability was clarified, with each change agreed with both stakeholders:

1. **Initial scope:** Control logic on a ctrlX CORE PLC with HMI, verified through full Hardware-in-the-Loop (HIL) with physical Bosch hardware.
2. **First revision (SIL):** The physical ctrlX CORE, drives, and motors were unavailable, so HIL was replaced with Software-in-the-Loop.
3. **Second revision (independent prototype):** SIL was skipped, and the team moved to its own hardware: an Arduino proof of concept, then an STM32H753 implementation.

Verification therefore shifted to MIL in Simulink and PIL on the STM32, which sends computed stroke lengths back to the Simscape plant over serial. Bosch tools (VMB, PLC Engineering, 3D Viewer) were retained for the industrial digital twin.

Consolidated scope

1. Build a working Stewart platform with its control algorithm as a functioning physical prototype.
2. Develop a digital twin in Simulink and the Bosch Rexroth 3D Viewer.
3. Verify the control system through MIL and PIL testing.
4. Conditional: integrate a web-based HMI via ctrlX CORE Virtual.

4.1.2 Performance metrics

The performance metrics are carried over from the Project Proposal and Completion Plan, then revised to match the final system. Since the ctrlX CORE runs only virtually and is not connected to the STM32, the communication-based metrics (EtherCAT drops < 5 %, dashboard latency < 40 ms) no longer apply and are removed. The original tracking error target (< 20 mm) is tightened into static pose and stroke accuracy targets, and the "zero steady-state drift" target between HIL and IK becomes a direct digital twin agreement metric. The final metrics are listed in Table 4-1.

Table 4-1: Performance metrics for the final system

ID	Metric	Target	Measurement method
PM-1	Heave accuracy	± 5 mm	Ruler/caliper from base to top platform
PM-2	Roll/pitch accuracy	± 2°	Phone inclinometer on top platform (1° resolution)
PM-3	Repeatability	Spread ≤ 2 mm / ≤ 1° over 5 trials	Repeat each pose 5 times from home
PM-4	Leg stroke accuracy	± 2 mm vs IK target	Caliper on each actuator extension
PM-5	Digital twin agreement	Twin vs physical pose within PM-1/PM-2 tolerances	Same command sent to Simscape and physical rig
PM-6	Workspace	± 20° roll/pitch and 0–150 mm heave reachable within the 0–150 mm stroke limit	Command boundary poses
PM-7	Physical envelope	Base ≤ 50 × 50 cm, height ≤ 100 cm	Tape measure
PM-8	Safety response	E-stop cuts driver power regardless of MCU state; STOP halts motion within ≈ 1 step	Functional test during motion

4.1.3 Design alternatives and selection

The main design decisions were evaluated with weighted decision matrices (Table 4-2 to Table 4-6). Each option is scored from 1 (poor) to 5 (excellent) against criteria derived from the stakeholder requirements (Section 4.1.1) and the performance metrics (Table 4-1); the weights sum to 1, and the highest weighted total is selected. The qualitative arguments behind the scores are given in Section 3.3. To check that each outcome does not depend on the chosen weights, every weight was varied by ±30 % (renormalised), and 5,000 random weight sets were sampled around the nominal values. The selected option is shaded in each table.

Table 4-2: Selection of the platform architecture

Option	Heave, roll and pitch capability (w = 0.30)	Stiffness and payload (w = 0.20)	Cost (number of actuators) (w = 0.20)	Kinematic and control simplicity (w = 0.15)	Compactness (w = 0.15)	Weighted total
Single-axis platform	1	3	5	5	4	3.25
2/3-axis gimbal	2	2	4	4	3	2.85
3-RPS parallel platform	**5**	**4**	**4**	**4**	**5**	**4.45**
6-DOF Stewart platform	5	5	2	2	4	3.80

The 3-RPS platform scores highest (4.45) because it is the only option that provides heave, roll and pitch with three actuators. The 6-DOF platform matches its capability but doubles the actuator count and control complexity for degrees of freedom the application does not need. The selection is unchanged under every weight variation tested.

Table 4-3: Selection of the actuator for the industrial build

Option	Positioning precision (w = 0.25)	Reliability (w = 0.20)	Cost (w = 0.25)	Ease of control and integration (w = 0.15)	Availability and lead time (w = 0.15)	Weighted total
RC servo + rack and pinion	2	1	5	5	5	3.45
Industrial servo cylinder	5	5	1	4	2	3.40
Bosch EMC cylinder + ctrlX DRIVE	5	5	1	4	1	3.25
Stepper electric cylinder	**4**	**4**	**4**	**3**	**3**	**3.70**

The stepper electric cylinder scores highest (3.70). The RC servo, used in the prototype for its low cost and ease of use, is penalised for reliability (two of six units failed), and the industrial servo and Bosch EMC cylinders for cost and availability. This is the most weight-sensitive decision: the stepper cylinder remains the choice in 71 % of sampled weight sets, with the RC servo winning only when reliability is de-weighted and the industrial servo only when cost is de-weighted.

Table 4-4: Selection of the main controller

Option	Real-time motor control (w = 0.25)	Ethernet / Bosch integration path (w = 0.20)	Availability to the team (w = 0.25)	Cost (w = 0.10)	Development effort (w = 0.20)	Weighted total
ctrlX CORE (physical)	5	5	1	1	2	3.00
STM32 Nucleo-H753ZI	**5**	**4**	**5**	**4**	**3**	**4.30**
ESP32	3	3	4	5	4	3.65
Arduino UNO	2	1	5	5	5	3.45

The physical ctrlX CORE is the best technical fit but scores 1 for availability because it was not supplied. The STM32 Nucleo-H753ZI (4.30) combines real-time motor control with an on-board RJ45 Ethernet port for a future ctrlX link, and remains the choice in 99.8 % of weight sets.

4.2 System architecture

The Stewart platform system consists of the HMI, controller, actuators, and plant. Each part also comprises an electrical design and a mechanical design.

4.2.1 Electrical and control architecture

The system centres on an STM32 microcontroller sitting between two input paths and two synchronised outputs. Users interact via either a direct PuTTY terminal or the IMU — alternative input modes, not parallel ones. The IMU is connected to the STM32 over I²C and provides orientation commands, while hardware START/STOP buttons wired into the STM32 GPIO manage the operating states. Independently of the MCU, a hardwired E-stop relay cuts the 24 V supply to the motor drivers. The Bosch HMI web app runs on ctrlX CORE Virtual and currently drives only the virtual actuators and the 3D Viewer.

Incoming commands are processed by four control modules on the STM32: a state machine, a drive algorithm, an inverse kinematics (IK) model, and a communication module. The STM32 also sends state updates back to PuTTY, keeping the operator informed of the actual system status rather than the last command issued.

From there, two outputs run simultaneously: STEP/DIR signals drive the DM542 motor drivers controlling the physical Stewart platform, while the computed stroke targets are mirrored over UART to Simulink for digital twin visualisation.

Defining this architecture explicitly gives every stakeholder a shared, unambiguous reference point — team members working on firmware, HMI, or simulation can develop against a common interface rather than assumptions, reducing integration issues and easing onboarding and design reviews.

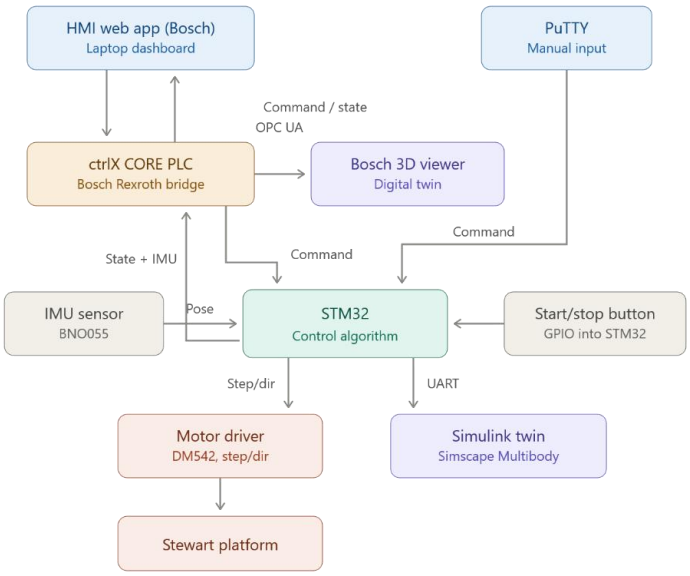


Figure 4-1: Electrical and control architecture

4.2.2 Mechanical architecture

The mechanism consists of three main components: the **Top Plate (end-effector)**, connected to **three Cylinders (actuators)** — each comprising two rigid links connected by a prismatic joint — and the **Base (reference frame)**. The three cylinders work in coordination to manipulate the position and orientation of the end-effector.

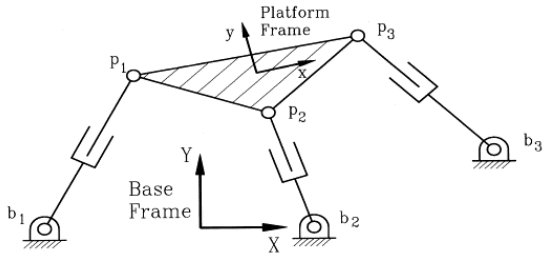


Figure 4-2: Mechanical architecture of the 3-RPS platform [14]

4.3 Stewart kinematics

As derived in the Capstone Part A report, inverse kinematics is the brain of the Stewart Model, as it calculates the required individual actuator lengths based on the user's inputs for Heave, Roll and Pitch.

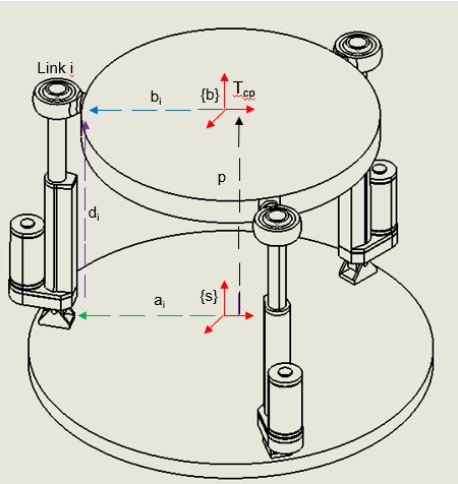


Figure 4-3: Closed-loop vector geometry of the 3-RPS platform [15]

The Stewart Platform is a closed-chain system. Figure 4-3 illustrates its structure and the closed-loop vector equation from various spatial vectors [15]:

$$d_i = p + R_{sb}b_i - a_i$$

Where:

- d_i: the resulting actuator vector from base anchor to platform anchor
- p: a 3D translational vector that maps the position from the center of the base frame {s} to the center of the moving platform {b}
- a_i: the static coordinate vector of the i-th base anchor joint, relative to the origin base frame {s}
- b_i: the static coordinate vector of the i-th platform anchor joint, relative to the platform {b}
- R_{sb}: the rotational matrix of frame {b} with respect to fixed frame {s} and $R_{sb} = R_x . R_y . R_z$ with R_z is a 3x3 identity matrix due to the yaw constraint in the design
- $R_{sb}b_i$: the moving platform vector joint transformed into the base orientation frame

The rotational matrix $R_{sb} = R_x(\zeta)R_y(\theta)R_z(\beta)$

$$R_{sb} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\zeta) & -\sin(\zeta) \\ 0 & \sin(\zeta) & \cos(\zeta) \end{bmatrix} . \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix} . \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ \sin(\zeta)\sin(\theta) & \cos(\zeta) & -\sin(\zeta)\cos(\theta) \\ -\cos(\zeta)\sin(\theta) & \sin(\zeta) & \cos(\zeta)\cos(\theta) \end{bmatrix}$$

Finally, the absolute length of the actuator (s_i) is:

$$s_i = \sqrt{d_i^T d_i} = \sqrt{d_{ix}^2 + d_{iy}^2 + d_{iz}^2}$$

Platform geometry. The three base revolute joints and the three platform spherical joints are each spaced 120° apart, at angles $\varphi_i = 0°, 120°$, and 240°. Their coordinates are:

$$a_i = [r_b \cos\varphi_i,\ r_b \sin\varphi_i,\ 0]^T, \quad b_i = [r_p \cos\varphi_i,\ r_p \sin\varphi_i,\ 0]^T$$

where r_b = 100 mm is the base joint radius and r_p = 100 mm is the platform joint radius (Table 8-2). Since yaw is constrained, the translation vector is $p = [0, 0, z_0 + h]^T$, where h is the commanded heave and z_0 = 0 mm is the platform height at the home position.

Stroke and step conversion. The stroke command for each actuator is the leg length minus its fully retracted length L_0 = 233.21 mm (Table 8-2), clamped to the 0–150 mm software range:

$$q_i = s_i - L_0, \quad 0 \le q_i \le 150 \text{ mm}$$

The Drive module converts the change in stroke into a step count $N_i = \text{round}(k \cdot (q_{i,\text{target}} - q_{i,\text{current}}))$, where k = (steps per revolution × microstep) / lead = 22.5 steps/mm.

4.4 Digital twin and verification strategy

4.4.1 Simscape digital twin

A digital twin is a digital representation of the Stewart Platform. It is intended to replicate the behaviour of the real Stewart Platform. A digital twin consists of three core elements: a physical asset, a virtual model, and a bidirectional data connection linking them together. The purpose of a digital twin is that we can validate the system before it is even built, mitigating the risk of broken hardware and lowering development cost. In this project, the data connection is one-directional (STM32 to Simscape); a bidirectional link is left for future work (Section 5.3). Under the classification of Kritzinger et al. [21], the Simscape and 3D Viewer models are therefore digital models driven by the controller's commands; feeding back measured leg positions would make them a digital shadow, and automatic data flow in both directions a true digital twin.

The standard digital twin development platform is MATLAB Simulink. The platform is open, has a lot of tools in its library, and offers suggestions for development. However, developing the whole system in a simulation platform creates intense computational demand, lowers the performance of the machine, and thus delays development. As a result, it cannot fully recreate the whole system, especially complex, expensive, high-performance machines such as those in the Bosch Rexroth ecosystem.

A better alternative is to leverage the Bosch Rexroth ecosystem. Since the system we are developing is from Bosch, using a Bosch software library dedicated to digital twins is much faster to gather and build with, as well as for verification within the system. The machines are already built by the company, so their behaviour and working principles should closely match.

Physical model and virtual model

As described in System Architecture, the Stewart Platform consists of the HMI, controller, actuators, and plant. In theory, to build a full digital twin, all of those aspects need to be designed and covered and thus simulated in a simulation platform such as MATLAB Simulink.

However, designing all of these aspects costs significant time and can even slow down the development of other aspects of the project, such as designing and building the actual physical platform. Thus, a decision was made to use the digital twin effectively, as part of early-stage modelling and verification, without spending too much time and research resources.

A simple HMI has been made using Constant and Slider blocks in Simulink. Another way to handle user input is through external user input, such as PuTTY or the HMI, which can be easily implemented using input blocks in Simulink Desktop Real-Time. We can take input from the Ethernet port or the UART port, based on our preferred communication protocol.

The controller is developed using a MATLAB Function that takes input from the Slider blocks and outputs the required stroke length for the prismatic joint of each linear actuator. The function takes the end-effector pose (pitch, roll, heave) as input and, using inverse kinematics, gives the required stroke travel.

In contrast, we can also model the actuators (motor driver and stepper motor) that receive the low-level signal commands (STEP, DIR, ENA) from the controller. This ensures maximum fidelity of the system and represents it at its fullest functional behaviour. However, after implementing this technique, we realised that it comes with major drawbacks. First, it significantly increases the compile time and execution time of the simulation, as modelling the electrical system requires more computational capability from the PC. Second, the time available to represent the complex nature of the stepper motor and driver was insufficient, so we were unable to implement this on time. Thus, the electrical system model is skipped to preserve development time and simulation time.

The plant is developed using Simscape Multibody. This tool is widely used as an industry standard to model the physical system of a plant, as it can handle the complex physical properties of a system. First, the model is created in CAD software,

then exported (via the Simscape Multibody Link plug-in) to an XML file that describes the bodies, joints, and frame coordinates of each link in text-based code, together with the part geometry files. MATLAB Simulink then imports this model and automatically converts it into a representative physical system in Simscape.

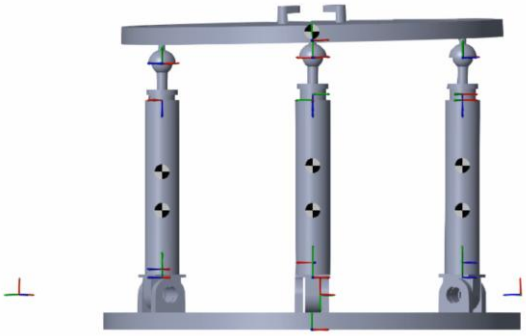


Figure 4-4: Simscape Multibody model of the Stewart platform

The physical system in Simscape Multibody contains the initial coordinates of the base and the physics of the world (gravity, etc.). It has the links and joints, connected to each other by the coordinate frames in each link. After import, the link size and shape cannot be changed; however, we can add more frames to add further joints. The joints are configurable in behaviour, for example maximum and minimum joint travel distance and force, and can be replaced by other kinds of joints. It is recommended that each joint be a true representation of the joint in real life, to ensure the system works as close to real life as possible.

The model follows the idea of simplicity discussed in the previous sections by receiving only the required stroke length for each joint. The stroke travel is then applied as an input to the prismatic joint in the plant model, where the force needed to move the joint is automatically computed in Simscape. The joints in Simscape can also measure the distance travelled, so the travel distance can very easily be fed back to the controller for closed-loop control, or measured with a Scope block, without the use of sensors.

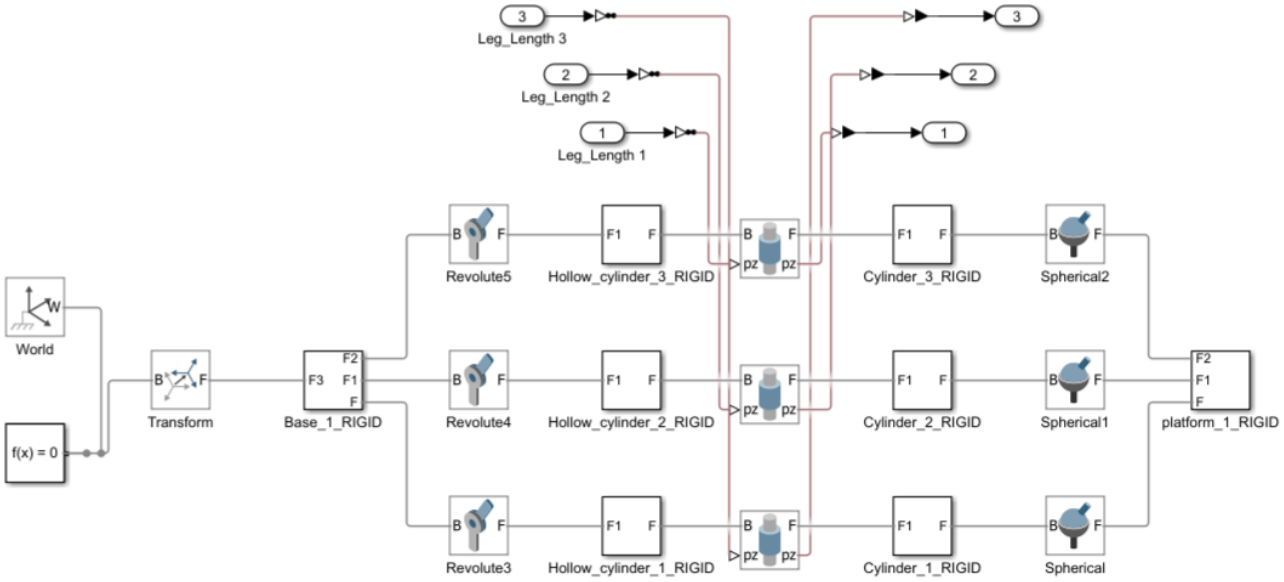


Figure 4-5: Simscape link, joint, and configurable joint parameters

The sensors of the system can also be modelled by external input blocks, such as SPI or I²C, but since the system is open-loop, this is not covered in the project.

With the controller and plant model developed, and the actuator and sensor models deliberately omitted, it is regarded as a sufficiently faithful kinematic representation of the platform.

Data communication link

The communication between the STM32 microcontroller and the Simscape Multibody plant model is established through UART serial communication using the Serial Configuration and Serial Receive blocks in Simulink, as shown in Figure 4-6.

The Serial Configuration block establishes communication through COM4 at a baud rate of 115200, using 8 data bits, no parity, and 1 stop bit (8N1). The Serial Receive block operates with a sample time of 0.01 seconds in non-blocking mode. When new data is unavailable, it retains the last received value.

On the STM32, the UART_SendLegLengths() function formats the three actuator stroke targets calculated by the inverse kinematics algorithm into an ASCII data frame and transmits it through USART2 using HAL_UART_Transmit(). The frame begins with the header leg\t, followed by three tab-separated floating-point values formatted to two decimal places, and ends with a carriage return and line feed (\r\n). The configured 26-byte payload contains three 8-character numerical fields and two tab separators.

The received data is passed to a processing subsystem, which extracts the three actuator stroke targets and supplies them to the corresponding prismatic joints of the Simscape Multibody plant model. This enables the virtual Stewart platform to respond to the actuator commands generated by the STM32.

The serial communication link supports Processor-in-the-Loop (PIL) testing by executing inverse kinematics on the physical microcontroller while retaining the Stewart platform plant in Simulink. The model receives actuator stroke targets directly, avoiding the additional computational complexity of simulating individual stepper motors and drivers.

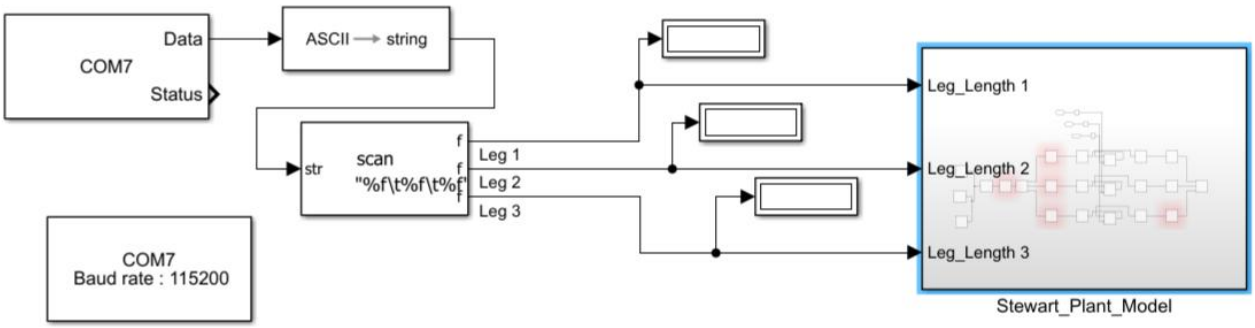


Figure 4-6: Serial Configuration and Serial Receive block parameters

4.4.2 Verification method

As referred to in the literature review, the verification method is a combination of processes: Model-in-the-Loop, Software-in-the-Loop, Processor-in-the-Loop, Hardware-in-the-Loop, and user acceptance testing.

In this project, MIL, PIL, and user acceptance testing are used. Model-in-the-Loop provides the initial logic planning for the algorithm and inverse kinematics, where a digital twin is developed alongside the control code, all in MATLAB/Simulink. After this stage, the algorithm is converted into C code to implement it on a real microcontroller. This process uses the Simulink code generation tool Embedded Coder. PIL further verifies the control on the microcontroller via UART; then, if the system works as intended, it is deployed to the real hardware.

The SIL process is not considered in the project because of its role as a middleman, rather than a crucial breakthrough in the development process. As for HIL, this very crucial process is unfortunately not used because of limitations in target machine devices (not provided) and the time-bound requirement, as the student team would need more time to learn how to use a target machine.

Model-in-the-Loop

First, the model is constructed as a digital twin, consisting of the MATLAB control function and the plant model. This process is very important to verify the development of the inverse kinematics and control logic of the system. From the user input, we expect the behaviour, which is seen in Mechanics Explorer in Simulink or the Bosch Rexroth 3D Viewer, to work as intended (correct pitch, roll, and heave movement and direction).

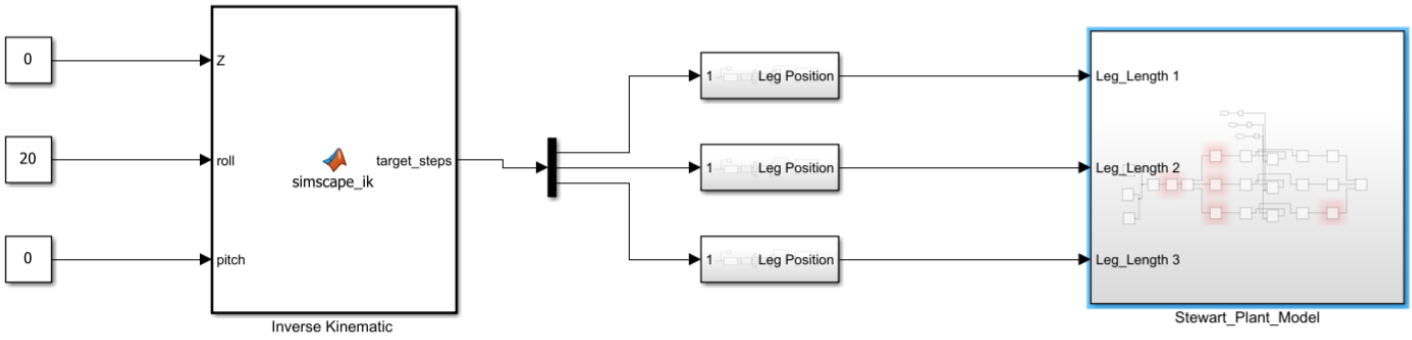


Figure 4-7: MIL test setup

Processor-in-the-Loop

After the control logic is tested and accepted, it is converted into C code for the microcontroller as the standard process, and the configuration of the STM32 pins to receive and transmit logic signals is implemented. This code can then be compiled and flashed onto the STM32 and debugged.

The main compiled code consists of input (end-effector orientation), processing (IK and control algorithm), and output (stroke travel length); however, at this stage the work was not yet finished, as the team still needed to add several modules, for example the HMI communication module (PuTTY/IMU) and the control module (state machine: Init and homing → Idle → START button → Running → STOP button → Stopped).

The main code outputs the stroke travel, which is then fed back into the plant model in Simulink using serial communication. As such, from the user input, the microcontroller directly processes the data, then sends command signals to the Stewart Platform plant model in Simulink Simscape.

As stated in the Simscape digital twin section, although the output signal for the digital twin model should ideally be low-level control for each motor driver, because of the complex nature of modelling electrical components in Simscape and to reduce simulation time, the MCU drives all 3 linear actuators in the Simscape plant model with stroke length only.

Field test

In parallel, the Drive module – low-level control for the stepper motor drivers – is also constructed. This module acts as a bridge between high-level control and low-level motor control. We use the data from the linear actuator, including pitch, travel speed, travel distance, and gear ratio, to determine the required motor travel to achieve the target stroke travel.

The digital twin and the physical platform are driven side by side from the same stroke commands, as shown in Figure 4-8. Their comparison is discussed in Section 5.1.1.

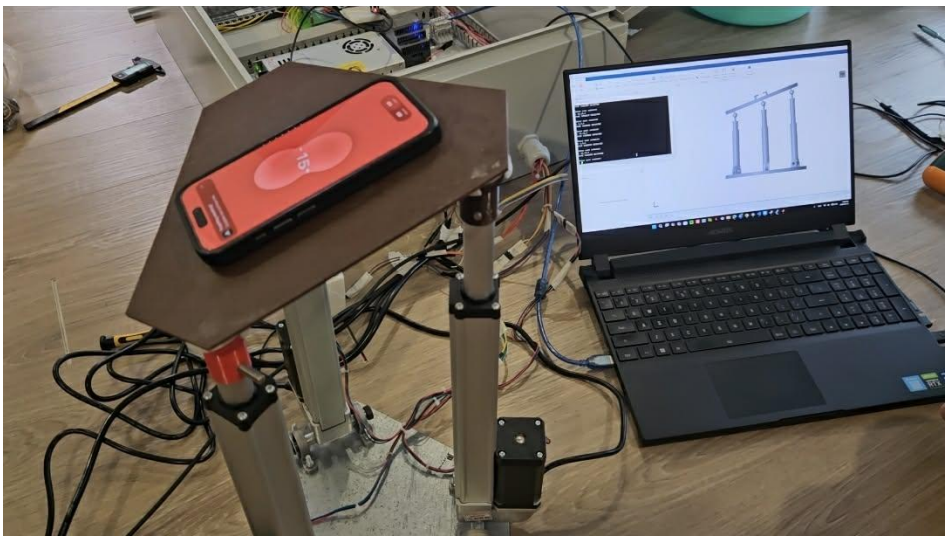


Figure 4-8: Parallel test of the physical platform and digital twin

4.5 Embedded implementation

4.5.1 Embedding code into the STM32

Firmware architecture and configuration

The physical Stewart platform is controlled by an STM32 NUCLEO-H753ZI microcontroller programmed in Embedded C. STM32CubeMX was used to configure the system clock, GPIO, I²C1, USART2, USART3, and TIM1 peripherals and to generate the initialisation code using the Hardware Abstraction Layer (HAL).

The firmware is organised into separate modules: main.c manages system initialisation, comms.c handles communication and sensor inputs, control.c manages operating states, ik.c calculates actuator stroke targets, and drive.c controls the stepper motors.

These modules were obtained by modularising the initial Embedded Coder output into readable, manageable, and distinct parts, with main.c reduced to peripheral initialisation, homing, and a single non-blocking call to Control_Update() in the main loop.

At startup, main() initialises the peripherals, starts TIM1, initialises the IMU, and performs actuator homing. The system then enters IDLE and repeatedly executes Control_Update() to process user inputs and control platform movement.

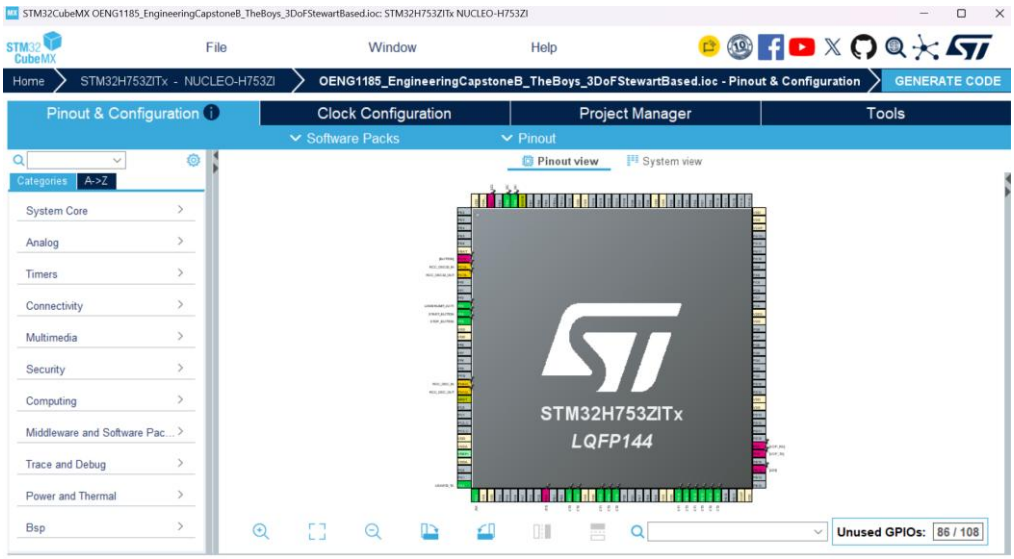


Figure 4-9: STM32MX Pins configuration

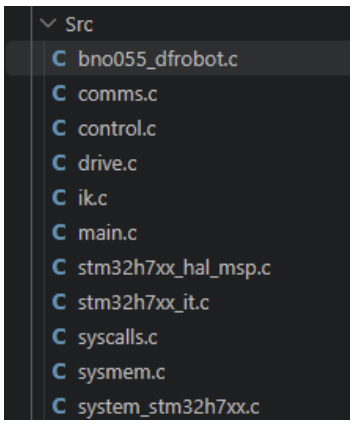


Figure 4-10: Modular organisation of the control algorithm

Actuator homing

The Drive_Home() function establishes a reference position for all three actuators before normal operation. The STM32 first sets the DIR pins to retract and generates STEP pulses simultaneously.

Each actuator has an individual lower-limit switch monitored through HAL_GPIO_ReadPin(). When a switch is activated, the corresponding motor stops receiving pulses while the remaining actuators continue retracting. Once all three switches are activated, their software-estimated positions are reset to 0 mm.

To prevent indefinite movement, homing is limited to a maximum commanded travel of 320 mm. Successful completion or failure is reported through PuTTY.

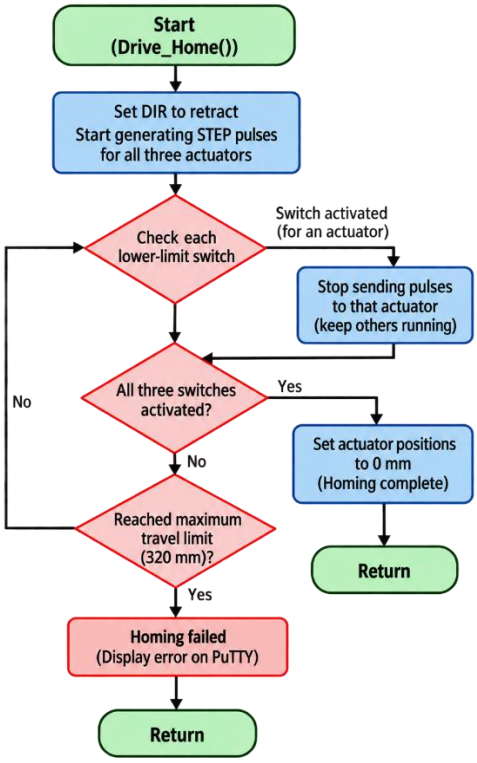


Figure 4-11: Homing procedure for the three linear actuators

Inverse kinematics and motor control

The simscape_ik() function converts the requested heave, roll, and pitch into three absolute actuator stroke targets. MoveActuatorsToTarget() calculates the displacement between each target and its estimated current position, then converts the displacement into the required step count.

Before the calculation, the inputs are clamped for safety: roll and pitch are each capped at 20°, and heave (Z) at 150 mm. Roll and pitch form the rotation matrix $R_{sb} = R_x \cdot R_y$ (Section 4.3), which is applied to the three platform joints spaced 120° apart. Each leg length is then computed as the 3D distance between the base and platform joints, converted to stroke, and clamped to a software range of 0–150 mm (within the 200 mm physical actuator travel).

The STM32 controls three DM542-05 stepper drivers using individual STEP and DIR GPIO signals. DIR determines extension or retraction, while each STEP pulse advances the motor by one configured step increment.

Stepper_Move3() generates pulses for all three actuators within a shared loop, allowing concurrent movement. Each motor stops receiving pulses after completing its required step count, and a shared abort flag is checked between pulses so that STOP halts motion within roughly one step.

The system clock is configured to 480 MHz. TIM1 provides a 1 MHz counter for microsecond delays, with nominal 500 µs HIGH and LOW STEP intervals. The firmware also updates the estimated actuator positions using the number of pulses actually issued.

IMU and PuTTY communication

The BNO055 IMU communicates with the STM32 through I²C1. During initialisation, the firmware enables NDOF sensor fusion, waits for gyroscope calibration, and averages 32 orientation samples to establish a zero reference. Comms_ReadIMU() then retrieves the roll and pitch angles.

PuTTY communicates through the ST-LINK virtual COM port using USART3 at 115200 baud. The Serial_ReadPoseCommand() function processes three command types: "10,5,-3" sets heave, roll, and pitch; "IMU" enables IMU-based control; and "A1,10,1" extends actuator 1 by 10 mm.

Incoming serial lines are buffered without blocking the execution flow, and each command is validated so that malformed frames are dropped. If the IMU fails its startup check, IMU reads are bypassed to avoid communication delays in the control loop.

In IMU mode, the sensor provides alternative orientation commands rather than feedback for correcting the physical platform position. A separate USART2 connection transmits the calculated actuator stroke targets to Simulink during manual pose commands.

Operating states and safety

The control.c module implements a four-state machine: INIT, IDLE, RUNNING, and STOPPED. The START and STOP buttons use external interrupts with software debouncing. Pressing STOP sets an abort flag that terminates the step-generation loop and moves the system to STOPPED, which prevents new movements until START is pressed. These buttons provide operational control only; emergency stopping is handled by a hardwired relay that cuts the 24 V supply to the DM542 drivers, independent of the firmware.

The button interrupt service routines only set flags; Control_Update() performs the state transitions in the main loop, keeping UART and motor calls out of interrupt context. STOP always takes priority over START.

The current motor-control implementation is open-loop: actuator positions are estimated from issued step pulses without encoder feedback.

4.6 Integration with Bosch

4.6.1 Bosch PLC implementation

There are 2 main segments in ctrlX CORE PLC Engineering: inverse kinematics computation and translational movement of the 3 linear actuators.

Inverse Kinematics Function Block

The derived inverse kinematics is integrated via a Function Block, as shown in Figure 4-12.

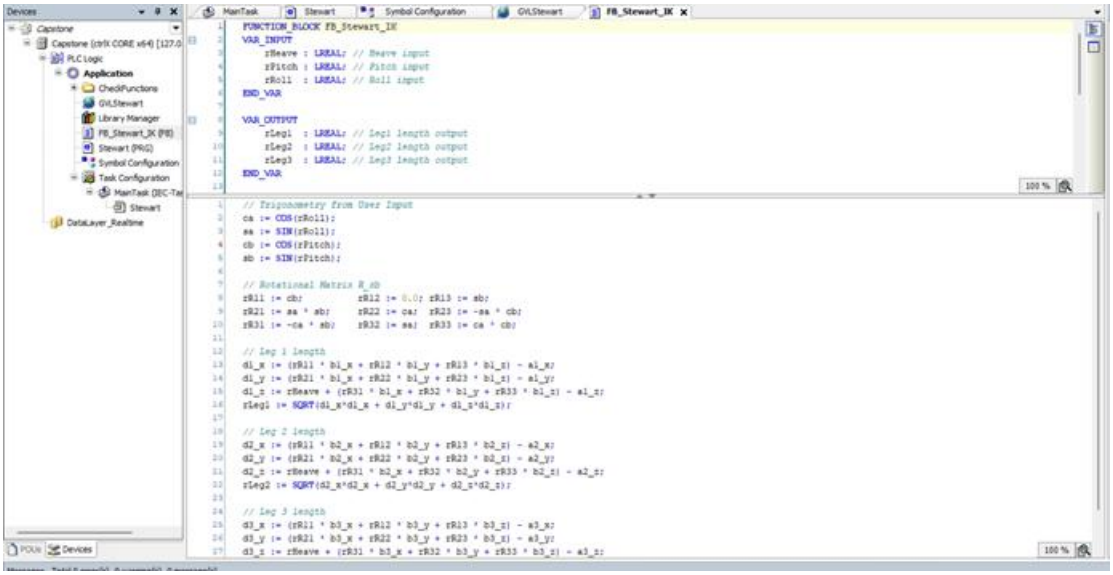


Figure 4-12: Inverse Kinematics Function Block in ctrlX PLC Engineering

Figure 4-13 illustrates all the variables in the Inverse Kinematics Function Block, including 3 user inputs (heave, pitch, and roll), 3 actuator lengths as outputs, and all trigonometric identities in the rotation matrix.

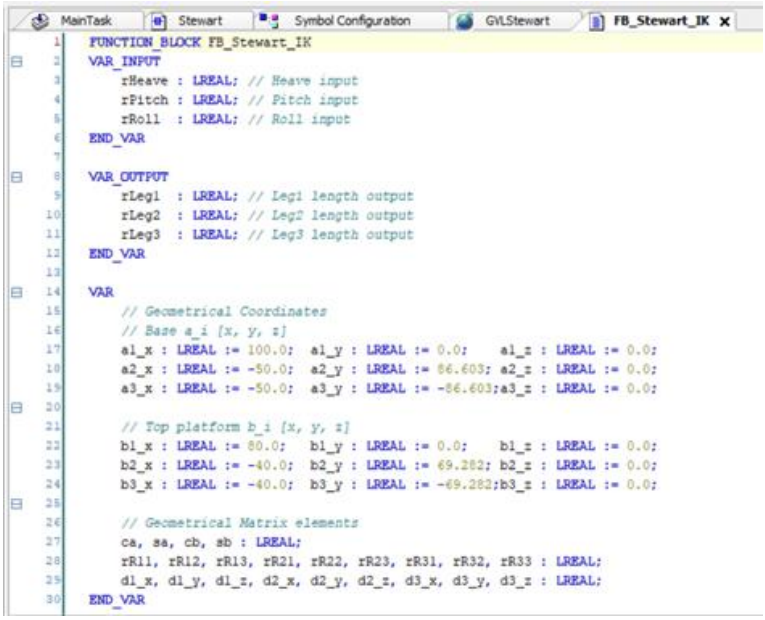


Figure 4-13: Variables of the Inverse Kinematics Function Block

Figure 4-14 demonstrates the implementation of the theoretical inverse kinematics formula in Section 4.3 using the variables in the ctrlX CORE controller and computes the actuator lengths to drive the platform to the required user inputs.

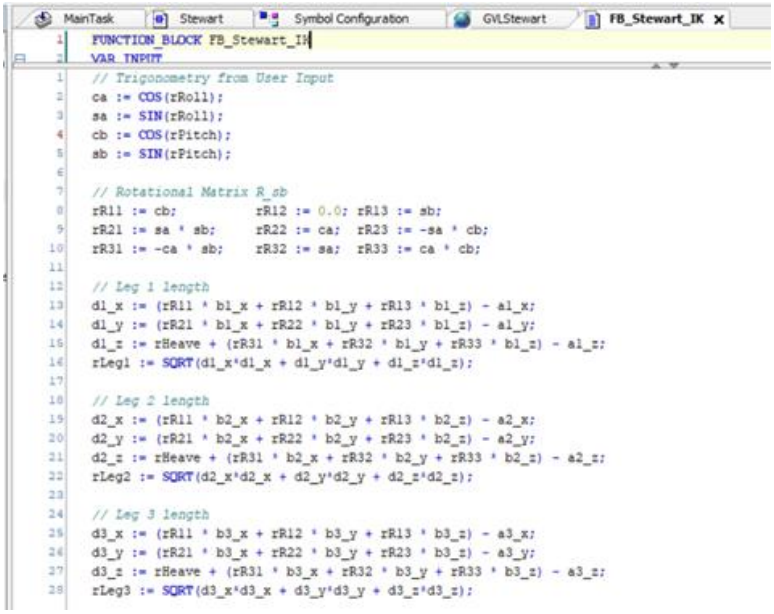


Figure 4-14: Implementation of the inverse kinematics in the ctrlX CORE

Actuator movement via Ladder Logic

Figure 4-15 illustrates the variables used for translational motion. *Motion Control (MC)* is embedded via the *CXA_PLCOPEN* library to control the virtual actuators, whereas *LastPos* stores the current position of the actuator.

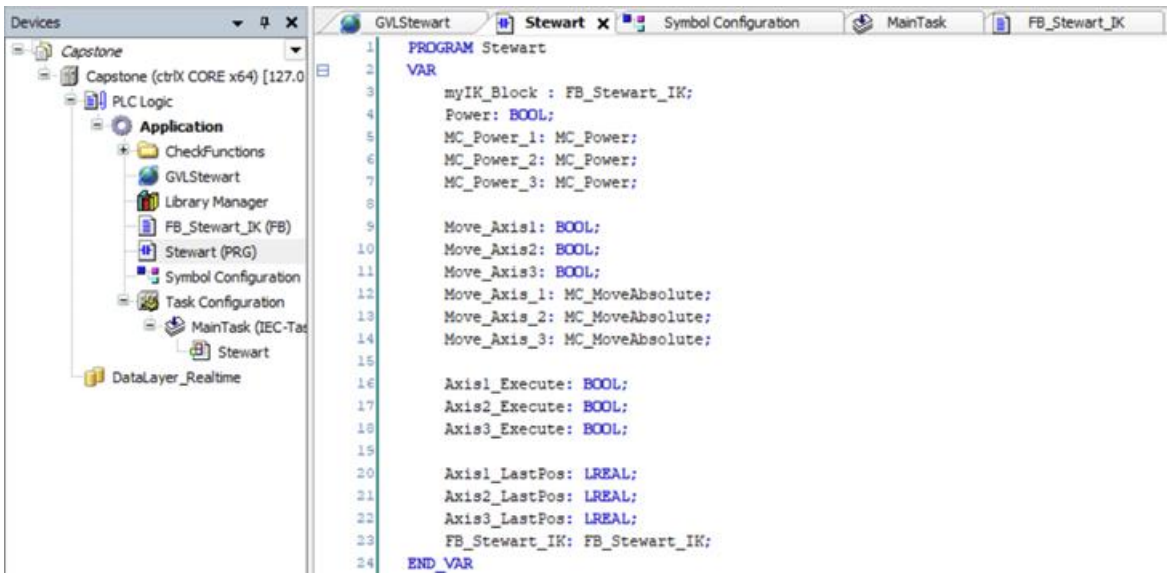


Figure 4-15: Variables used for translational motion

The pre-developed Inverse Kinematics Function Block is implemented in the Ladder Logic main program with 3 inputs and 3 outputs, as shown in Figure 4-16.

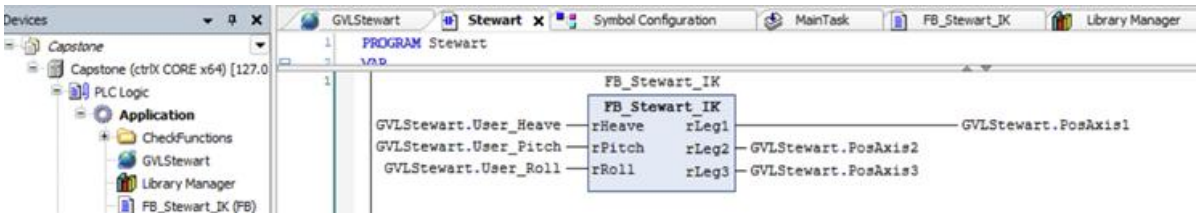


Figure 4-16: Inverse Kinematics Function Block in the Ladder Logic main program

Figure 4-17 demonstrates the complete control of axis 1. *MC_Power* and Move_Axis_1 activate *Motion Control* and *MC_MoveAbsolute*, respectively. The *EXECUTE BLOCK* checks for the presence of a new position to trigger the edge-detection contact *Axial_Execute*. Then, *MC_MoveAbsolute* drives the actuator to the calculated position via the global variable PosAxis1 with the preset kinematic parameters shown.

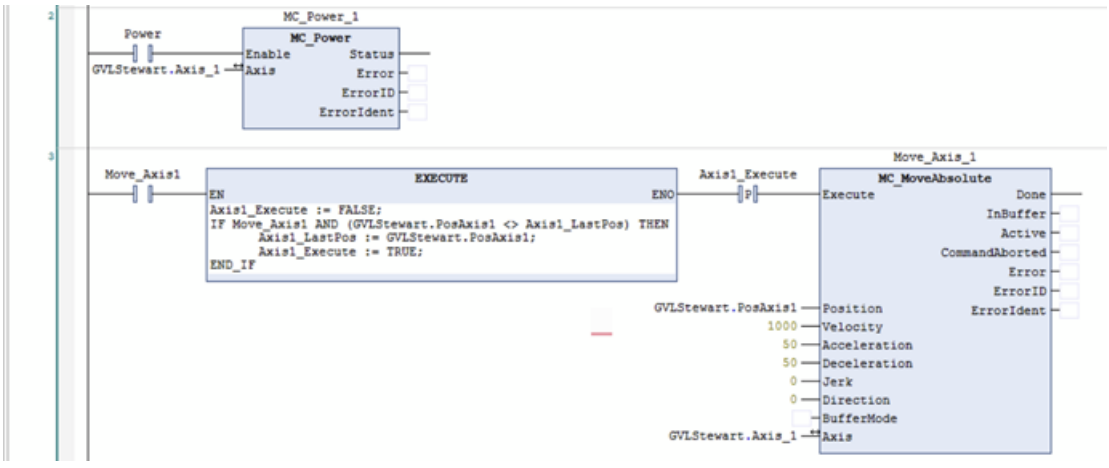


Figure 4-17: Ladder Logic control of axis 1

4.6.2 Monitoring using HMI

HMI implementation

The HMI is built with HELIO, a web-based HMI tool from HMI Project GmbH that runs as an app within the ctrlX OS ecosystem and connects to the controller over OPC UA [16]. In this project, HELIO runs on ctrlX CORE Virtual alongside

the ctrlX OPC UA Server app. An OPC UA data source is configured in HELIO's Data Source Explorer, and the PLC global variables are then bound to HMI elements without additional programming.

The HMI reads and writes three groups of variables:

- Inputs (HMI → PLC): the target pose GVL.heave_user, GVL.roll_user, and GVL.pitch_user, entered through numeric fields, together with the start/stop commands and the control mode selection (manual or IMU)
- Outputs (PLC → HMI): the three actuator lengths calculated by the Inverse Kinematics Function Block and the current axis positions (LastPos), shown as live stroke readouts.
- States: power on/off, IDLE, and WORKING, displayed so the operator can see the current system state before issuing a command.

Because ctrlX CORE Virtual is not connected to the STM32, the HMI currently commands the virtual axes and the 3D Viewer only; the same variable interface is intended to drive the physical platform once the STM32–ctrlX link is implemented.

Purpose of the HMI

In this system, the HMI acts as the operational bridge between the user and the ctrlX CORE controller. It translates high-level task coordinates (heave, pitch, and roll) into controller memory tags without requiring direct access to the PLC development environment.

The interface serves four main functions:

- Setpoint entry: Accepting target angles and height adjustments for manual positioning, or switching to dynamic IMU tracking.
- State sequencing: Commanding operational transitions between IDLE and WORKING states through start and stop triggers.
- Actuator monitoring: Displaying live stroke extensions for all three linear legs alongside calculated joint angles.
- Visual validation: Rendering a real-time 3D model that reflects the platform orientation only when the commanded pose is mathematically valid.

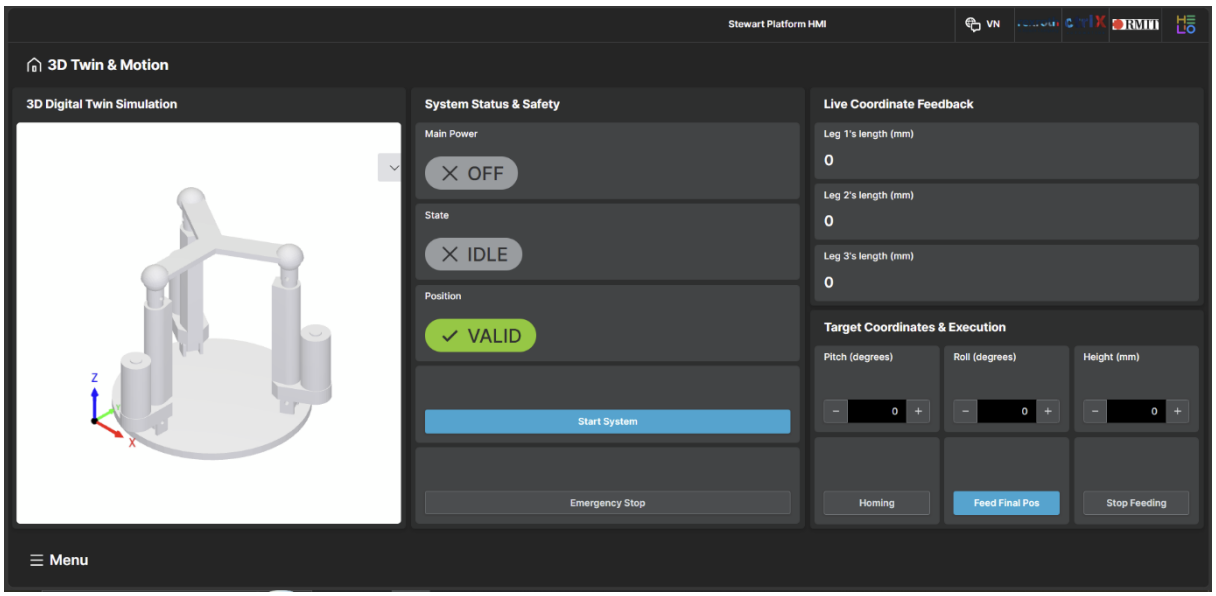


Figure 4-18: Web-based HMI for the Stewart platform

Benefits of using an HMI versus operating without one

Operating a parallel robot without an HMI requires an engineering workstation running proprietary programming software, such as ctrlX PLC Engineering. An operator would need to open variable watch tables, locate internal memory tags (GVL.heave_user, GVL.roll_user, GVL.pitch_user), and manually force values into the PLC runtime.

Using an HMI provides three distinct functional advantages over raw PLC operation:

1. Workspace boundary protection before actuation

Parallel manipulators possess non-linear kinematics. A small change in pitch or roll can drive one actuator past its physical limit while the other two remain well within range. Operating directly through PLC watch tables leaves the user blind to these boundaries until an actuator hits a physical limit switch or trips a drive error.

2. Spatial awareness through kinematic visualisation

Raw numerical readouts in a PLC table (such as millimetre lengths or radian values) do not provide an intuitive picture of mechanical tilt, especially with the coupled leg motions of a 3-RPS platform. The WebGL digital twin translates inverse kinematic calculations into an immediate 3D pose. By retaining the previous valid pose when invalid targets are entered, the interface prevents visual discrepancies between the screen and the physical machine.

3. Operational abstraction and access independence

Controlling the platform through an IDE exposes the control logic to accidental code modifications and requires specialised software licences. The HELIO HMI communicates over standard OPC UA protocols and renders in any web browser. Operators can command and inspect the machine from any networked terminal without installing automation engineering tools or accessing the underlying PLC logic.

System boundaries and limitations

The HMI improves operational usability, but it does not replace low-level controller tasks. HMI updates rely on asynchronous network polling over OPC UA, which introduces communication latency. Trajectory generation, position control of the virtual axes, and limit enforcement remain executed inside the PLC task cycle, while physical safety stops rely on dedicated electrical circuits rather than web controls.

4.6.3 3D Viewer digital twin

The 3D Viewer digital twin is developed with Bosch Rexroth VMB (Virtual Machine Builder) and ctrlX PLC Engineering.

XML file generation

Creating transform nodes:

- **Rotation nodes** are used for the spherical and revolute joints.
- **Translation nodes** are used for the prismatic joints.

The transformation hierarchy follows this order:

Base revolute joint position → Revolute joint rotation → Prismatic joint translation → Spherical joint rotation → Top platform roll and pitch

Therefore, the rotational nodes are parent nodes of the prismatic nodes, while the prismatic nodes are parent nodes of the final top-platform roll and pitch.

After all nodes are defined, the XML file is exported from the VMB.

Important: For the spherical joints, the Euler angle hierarchy must follow the correct rotation order. The **pitch node must be the parent of the roll node**, as shown in the XML hierarchy.

3D Viewer

The exported XML file is combined with the STL files of each Stewart platform component for the 3D Viewer.

Each transform node is then linked to Bosch PLC variables through the Data Layer.

However, the transform nodes operate independently, so simply extending an actuator does not correctly move the platform. Therefore, **inverse kinematics** is used to calculate the revolute joint angles, leg lengths, and top-platform roll and pitch. These PLC variables are then made accessible through the Data Layer.

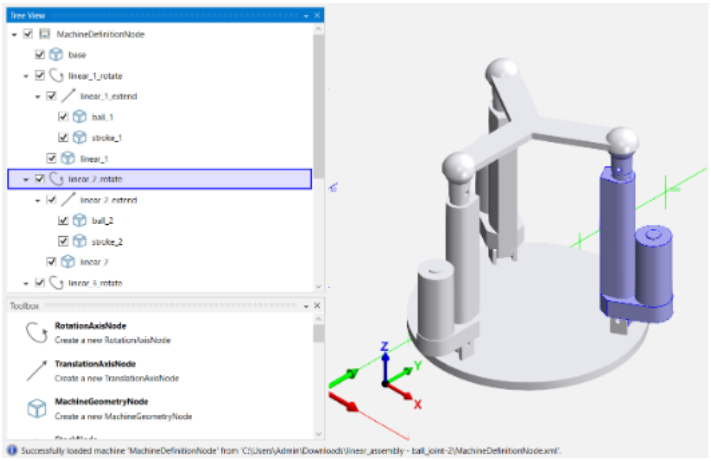


Figure 4-19: Stewart platform digital twin in the Bosch Rexroth 3D Viewer

4.7 Prototype design

4.7.1 Arduino working prototype

A 3D-printed working prototype using an Arduino controller is constructed to validate the inverse kinematics derived in Section 4.3, its dimensions are listed in Table 4-7.

The rack-and-pinion mechanism is used to transform the rotational motion of the servo motors into linear motion via the equation:

$$s = r \cdot \theta$$

where θ is the gear rotation angle (in radians), s is the linear displacement (in mm), and r is the gear radius (in mm).

The MG996R servo motor controls the gear's rotational motion. To enable bidirectional rack movement within the motor's 0° to 180° operating range, the home position is calibrated to 90°.

Specifically, the gear radius is approximately 35.5 mm, so the full 180° servo rotation gives a theoretical travel of $\pi \times 35.5 \approx 111.5$ mm. After the mechanical clearances of the 3D-printed rack assembly, the usable travel is limited to 100 mm (Table 4-7), split equally between upward and downward movement, as shown in Figure 4-20.

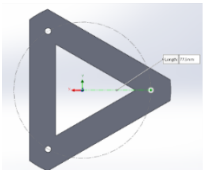
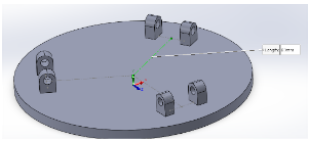
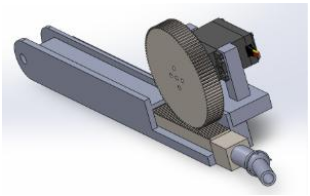


Figure 4-20: Rack-and-pinion travel range of the prototype actuator

4.7.2 Working prototype Stewart platform

The Stewart platform uses **custom linear actuators with rack-and-pinion mechanisms** to simulate the real actuators. This model helps us understand the platform's real-world movement before implementing the actual linear actuators.

Table 4-5 Components of the working prototype

Component name	Description	Figure
Top platform	The spherical joints are **77.5 mm from the centre** of the top platform.	
Base platform	Contains **3 revolute pins**, equally spaced **120° apart** in a circle. Each pin is **80 mm from the centre**.	
Rack assembly	Acts as the custom linear actuator. An **MG996R servo rotates the gear**, which moves the rack. The spherical joint is attached to the end of the rack. At the home position, the distance from the revolute pin to the spherical joint is **273.5 mm**, with a total travel distance of **100 mm**.	

4.8 Industrial design

The industrial version replaces the servo prototype with three stepper-driven linear actuators (200 mm travel, software-limited to 150 mm) with ball-joint tips, mounted to a laser-cut base through KP08 pillow bearings acting as revolute joints, and a top platform with a 100 mm joint radius. The control panel houses a 24 V Mean Well LRS-350-24 supply for the three DM542 drivers, a 5 V Mean Well HDR-30-5 supply for the STM32 and logic, a 16 A two-pole circuit breaker, the START/STOP buttons, and the hardwired E-stop relay that cuts the 24 V driver supply. The wiring diagram and full component lists are given in Appendix 8.2 (electrical) and Appendix 8.3 (mechanical).

5 Result analyses & discussions

5.1 Result analysis

As referred to in the scope of work, the resulting deliverables are:

1. Design and build a working Stewart platform together with its control algorithm, and deliver a functioning physical prototype.
2. Develop a digital twin of the Stewart Platform in Simulink and the Bosch Rexroth 3D Viewer.
3. Apply Model-in-the-Loop (MIL) and Processor-in-the-Loop (PIL) testing to verify the control system.
4. Conditional: integrate a web-based HMI to control the Stewart Platform through ctrlX CORE Virtual.

In this project, deliverables 1, 2, and 4 have been achieved, and deliverable 3 is partially achieved (Table 5-3). It should be noted, however, that the connection pipeline between the STM32 and the ctrlX CORE is not completed due to time constraints.

Furthermore, we were able to extend the deliverables by applying the IMU sensor as a user input for interaction. Another significant extension is the implementation of industrial equipment, from the control panel to industrial linear actuators and rigid connection links, eliminating much of the mechanical slop.

The physical Stewart platform was evaluated through three positioning tests involving heave, roll, and pitch. Each pose was tested three times. The actual heave was measured using a ruler or caliper, while roll and pitch were measured using a phone inclinometer. The commanded and measured values are presented in Table 5-1.

Table 5-1: Comparison between commanded and measured platform poses

Commanded (heave, roll, pitch)	Measured (heave, roll, pitch)	Absolute error
100 mm, 0°, 0°	100 mm, 0°, 0°	Heave: 0 mm
100 mm, 20°, 0°	100 mm, 19°, 0°	Roll: 1°
100 mm, 0°, 20°	100 mm, 0°, 20°	Pitch: 0°

All three poses were reached within 0 mm and 1° of the commanded values (Table 5-1); the only deviation was a 1° shortfall in the 20° roll test.

These results provide initial evidence that the physical platform can execute individual heave, roll, and pitch commands. However, the evaluation was limited to three operating points, with three trials per pose.

5.1.1 Digital twin versus physical platform

To evaluate PM-5, the same pose commands are applied to the Simscape Multibody digital twin and to the physical platform. The STM32 computes the actuator stroke targets and sends identical values to both: over USART2 to Simulink, and as STEP/DIR pulses to the DM542 drivers. The twin pose would be read from a Simscape transform sensor on the top platform, and the physical pose measured as described above. Because both sides receive the same stroke targets, any difference between them isolates the physical error sources (missed steps, joint play, mechanical slop) from the kinematics itself.

During PIL testing, the two were run side by side from the same commands (Figure 4-8), and both moved in the commanded directions. However, because the linear actuators arrived the weekend before Demo Week, the Simscape pose and the physical leg strokes were not logged, so no quantitative comparison could be made within the project timeframe. PM-4 and PM-5 are therefore not verified, and this comparison is carried forward as the first item of future work (Section 5.3.4).

5.2 Performance evaluation

The performance evaluation focused on the positioning accuracy of the physical Stewart platform by comparing commanded and measured poses.

Table 5-2 summarises the results against the performance metrics defined in Table 4-1.

Table 5-2: Evaluation against the performance metrics

ID	Target	Result	Status
PM-1	± 5 mm	0 mm error at 100 mm (3 trials per pose)	Met

ID	Target	Result	Status
PM-2	± 2°	Roll 1°, pitch 0° error at 20° (3 trials per pose)	Met
PM-3	≤ 2 mm / ≤ 1° spread over 5 trials	3 trials per pose; spread between trials not recorded	Not verified
PM-4	± 2 mm	Leg strokes not measured	Not tested
PM-5	Within PM-1/PM-2	Twin pose not logged (Section 5.1.1)	Not tested
PM-6	± 20° roll/pitch, 0–150 mm heave	20° roll and pitch reached; heave tested at 100 mm only	Partially met
PM-7	≤ 50 × 50 cm, ≤ 100 cm	measure base footprint and height	Not tested
PM-8	Power cut / ≈ 1-step stop	Hardwired E-stop relay implemented; stop response not formally timed	Not tested

The heave measurements showed zero reported error at the tested height of 100 mm. The roll test recorded an absolute error of 1°, corresponding to 5% of the commanded 20° angle. The pitch test recorded no error at the reported measurement precision.

These measurements indicate small deviations at the tested positions. However, the results must be interpreted with consideration of the measurement methods.

Heave was measured manually using a ruler or caliper, while orientation was measured using a phone inclinometer (iPhone level). The accuracy, resolution, and calibration of these measurement methods were not independently verified. Therefore, a reported error of zero does not necessarily indicate perfect positioning.

The 1° roll error is within PM-2 but equals the resolution of the phone inclinometer, so it cannot be separated from measurement error. It also occurred at 20°, the firmware clamp limit, where the actuators are furthest from their home positions. Possible physical sources are joint play in the ball joints and pillow bearings and missed steps under the higher leg load at large tilt, which the open-loop drive cannot detect. Overall, PM-1, PM-2, and the tested part of PM-6 are met, while the metrics that require additional instrumentation (PM-3, PM-4, PM-5, PM-8) remain unverified.

5.3 Discussion

5.3.1 Achievement of objectives

Table 5-3 compares each deliverable from the consolidated scope with its outcome.

Table 5-3: Deliverables versus outcomes

Deliverable	Outcome	Status
Working Stewart platform with control algorithm	Industrial platform built with STM32 control, homing, IMU mode, and hardwired E-stop; pose tests within PM-1/PM-2	Achieved
Digital twin in Simulink and the Bosch 3D Viewer	Simscape twin driven by STM32 strokes; 3D Viewer twin driven by the ctrlX PLC; the two twins are not linked to each other	Achieved (open-loop)
Verification through MIL and PIL	MIL in Simulink and PIL over UART completed; quantitative twin-vs-physical comparison not recorded	Partially achieved
Conditional: web-based HMI via ctrlX CORE Virtual	HELIO HMI over OPC UA controls the virtual axes; not connected to the physical platform	Achieved (virtual only)

5.3.2 Project timeline

The project schedule is shown in the Gantt chart in Figure 8-2. It spanned twelve weeks from 30 June to 19 September 2026 and was organised into five parallel streams: project management and architecture, control algorithm and code generation, ctrlX CORE PLC programming, HMI development, and hardware integration.

The software streams largely followed the plan. The system architecture and task allocation were completed by mid-July, and the MATLAB/Simulink kinematics and control design by 25 July. The ctrlX PLC work started with the manual study and continued into the inverse kinematics implementation and motion control verification during August, while the HMI

layout was developed from early July. Because these streams depended only on software, they progressed independently of the hardware delays.

The main deviations came from the hardware-dependent tasks. The chart still reflects the Completion Plan: a cascaded closed-loop PID controller, SIL/PIL/HIL validation, and PLC-based drive integration. When the physical ctrlX CORE was confirmed unavailable in late July (Meeting 4), the team replaced it with an STM32 purchased in early August. As a result, the PID controller gave way to open-loop stepper control, SIL was skipped, and the "real-time communication between the PLC and the HIL target" task was not carried out. The hardware integration stream (mechanical setup, control panel, and electrical and drive integration) was planned for 12 August to 16 September, but the linear actuators arrived only the weekend before Demo Week, compressing about four weeks of integration and calibration into a few days.

The key lesson is that hardware procurement sat on the critical path with no schedule buffer. In future projects, long-lead items should be ordered in the first weeks, and a fallback plan should be agreed for any externally supplied hardware.

5.3.3 Risk assessment

This assessment applies the hazard evaluation principles of ISO 12100:2010 to evaluate the physical test bench, embedded control pipeline, and commissioning schedule [17]. Risk Priority Levels (RPL) are scored as Severity (1 to 4) multiplied by Probability (1 to 4).

Mechanical kinematics and structural integrity

The platform uses open-loop stepper motors without encoder feedback. In a parallel kinematic architecture such as a 3-RPS mechanism, the three linear actuators are mechanically coupled. If an actuator loses steps under load, the physical leg length deviates from the internal step counter on the STM32H753. As J.-P. Merlet details in *Parallel Robots* (Springer, 2006), parallel platforms generate severe constraint forces near kinematic boundaries, which can twist lead screws and destroy ball joints [18]. Relying on deformable plastic joints as a mechanical fuse has limitations: Plastic suffers from progressive creep and fatigue under cyclic loading. This wear creates joint play and invalidates the inverse kinematics calculations. To prevent mechanical lockup, the firmware clamps platform pitch and roll to 20 degrees each, heave to 150 mm and actuator stroke to 0–150 mm, while lower-limit switches enforce mechanical zeroing on every boot cycle.

Functional safety and electrical interference

Routing the emergency stop only through an STM32 GPIO pin (EXTI) would create an unacceptable safety risk. Under ISO 13849-1:2023 and IEC 60204-1:2016, an emergency stop cannot depend on software execution [19], [20]. If the microcontroller locks up or encounters a hard fault, software-gated stops fail completely. Safe operation requires a hardwired circuit to sever main DC power to the DM542 stepper drivers immediately (Stop Category 0). Therefore, the START/STOP buttons on GPIO are used only for operational state control, while a hardwired safety relay cuts the 24 V supply to the drivers. Additionally, high-frequency PWM switching from the stepper drives induces electromagnetic noise on adjacent signal lines. Unshielded cabling routed near the BNO055 IMU risks I²C bus lockup, which stalls the control loop unless non-blocking bus timeouts are enforced.

Simulation decoupling and deployment schedule

The physical Hardware-in-the-Loop (HIL) pipeline was abandoned because Bosch Rexroth could not supply the physical ctrlX CORE hardware. The ctrlX virtual model and its 3D Viewer operate on an isolated track without closed-loop feedback from the physical rig. The virtual twin displays calculated values rather than verified physical positions. It therefore functions as an open-loop supervisory visualiser rather than a closed-loop industrial digital twin; adding physical feedback is future work (Section 5.3.4). Compounding this, the physical actuators arrived the weekend before Demo Week, which eliminated the window needed for proper commissioning, load testing, and kinematic calibration.

Table 5-4: Risk mitigation action matrix

Risk ID	Hazard and root cause	Initial (S×P)	Core engineering mitigation	Final (S×P)
M-01	Actuator binding from missed steps (open-loop)	3 × 4 = 12 (High)	Clamped pose boundaries (20° roll/pitch, 150 mm heave, 0–150 mm stroke); boot homing routine.	2 × 2 = 4 (Med)
M-02	Joint fatigue and play from plastic components	3 × 3 = 9 (High)	Velocity and acceleration limits; routine checks for mechanical wear.	2 × 2 = 4 (Med)
E-01	Software E-stop failure during microcontroller lockup	4 × 2 = 8 (Med)	Hardwired 24 V power cut to DM542 drivers via safety relay (IEC 60204-1 Stop Cat. 0) – implemented.	2 × 1 = 2 (Low)

Risk ID	Hazard and root cause	Initial (S×P)	Core engineering mitigation	Final (S×P)
E-02	EMI bus freeze on BNO055 I²C sensor lines	2 × 3 = 6 (Med)	Physical line separation; 2.2 kΩ pull-ups; non-blocking timeout routines.	1 × 2 = 2 (Low)
C-01	Divergence between ctrlX virtual model and reality	2 × 4 = 8 (Med)	Reclassify 3D Viewer as open-loop display; enforce limits locally on STM32.	2 × 2 = 4 (Med)
P-01	Compressed commissioning from late actuator arrival	3 × 4 = 12 (High)	Staged low-speed testing; prior logic verification on RC servo prototype.	2 × 2 = 4 (Med)

5.3.4 Limitations and future work

The main limitations of the final system, and the recommended next steps, are:

- Quantitative validation: the twin-vs-physical comparison (PM-5), leg stroke accuracy (PM-4), and five-trial repeatability (PM-3) were not measured. The first next step is to log the Simscape pose and caliper strokes for the same commands, and to automate pose measurement with the BNO055 and a distance sensor.
- Open-loop actuation: step loss cannot be detected. The built-in Hall encoders of the cylinders can be used to close the position loop on each leg.
- Decoupled Bosch integration: the ctrlX CORE Virtual, HMI, and 3D Viewer are not linked to the STM32. Connecting them over OPC UA through the Nucleo's Ethernet port would give a single control and monitoring path.
- Digital twin fidelity: the Simscape twin omits the motor and driver models and receives data in one direction only. Adding feedback from the physical rig would turn it into a bidirectional digital twin.
- HIL coverage: full HIL testing still requires a real-time target machine and the physical ctrlX CORE.
- Safety validation: the hardwired E-stop is implemented but its response was not formally tested (PM-8).

6 Summary and reflection

To begin with, we have broadened our minds throughout Capstone Parts A and B, from technical knowledge to management skills. It was also an opportunity for our group to discover real-world problems. First of all, we had the chance to work with Bosch – an established corporation in the industry. We discovered what Hardware-in-the-Loop (HIL) is and worked with the Bosch PLC – ctrlX CORE. We managed to control and display the Stewart Platform in the 3D Viewer via a Function Block and Ladder Diagram. However, we did not accomplish HIL because a physical ctrlX CORE controller was not provided. We understood that hardware availability was constrained on the company side. Alternatively, we purchased an STM32 to compensate for the lack of the Bosch controller, as an industrial system was preferred. Even though the programming was challenging at the time and, especially, the three linear actuators arrived on the weekend before Demo Week, the final Stewart Platform, including the simulation, prototype, and final product, worked moderately well.

Furthermore, all of us learnt new and beneficial knowledge: kinematics modelling, PLC programming, CAD design, HMI configuration, embedded programming, electrical layout, and developing control algorithms. On the project management side, even though we were all busy with internships, other courses, and assignments, we still managed to cover the key aspects and technical concepts, apart from some details. However, our collaboration and proactive communication are key areas for improvement. To be more specific, a weekly presentation with three main parts – *What have we done so far*, *What are the current issues*, and *What are the plans for next week* – would be helpful and informative for both the academic and industry supervisors to monitor progress and address any urgent problems.

Overall, the two parts of Capstone 2026 have enhanced our technical knowledge, problem-solving skills, and project management. Reflecting back, it was a valuable lesson for us to prepare actively for the project, plan the schedule appropriately, manage all the risks that can affect completion progress, and propose flexible mitigation actions. In conclusion, the Capstone Project was a worthwhile maturing experience for us, as we gained technical knowledge, collaborated with Bosch, resolved various issues, made mistakes, and learnt from them. These experiences will serve as a solid foundation for our team in future career opportunities.

7 References

[1] J. Moura and D. Hutchison, "Review and analysis of networking challenges in cloud computing," *Journal of Network and Computer Applications*, vol. 60, pp. 113–129, Jan. 2016, doi: 10.1016/j.jnca.2015.11.015.

[2] Bosch Rexroth AG, "Bosch Rexroth and MathWorks combine simulation and automation," press release. Accessed: Sep. 22, 2026. [Online]. Available: https://www.boschrexroth.com/en/de/company/press/bosch-rexroth-and-mathworks-combine-simulation-and-automation-13312.html

[3] Software Defined Automation, "Secure remote access," Hannover Messe exhibitor product page. Accessed: Sep. 22, 2026. [Online]. Available: https://www.hannovermesse.de/product/sicherer-fernzugriff/444880/N1504881

[4] Motor_Explorer1, "CRAZY Machine Can Balance ANYTHING!?," *YouTube*. Accessed: Sep. 22, 2026. [Online Video]. Available: https://www.youtube.com/shorts/sEeIYPTgji4

[5] maxon group, "The docking system for safe space rendezvous," *maxon Blog*. Accessed: Sep. 22, 2026. [Online]. Available: https://www.maxongroup.com/en-gb/knowledge-and-support/blog/the-docking-system-for-safe-space-rendevous-23804

[6] "Full flight simulator with the Stewart platform," ResearchGate. Accessed: Sep. 22, 2026. [Online]. Available: https://www.researchgate.net/figure/Full-flight-simulator-with-the-Stewart-platform_fig1_371793479

[7] "Stewart platform," *Wikipedia*. Accessed: Sep. 22, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Stewart_platform

[8] Clean Aviation, "Clean Sky's POLITE model gets its second wind." Accessed: Sep. 22, 2026. [Online]. Available: https://clean-aviation.eu/clean-skys-polite-model-gets-its-second-wind

[9] Exail, "Stabilized platforms." Accessed: Sep. 22, 2026. [Online]. Available: https://www.exail.com/product-family/stabilized-platforms

[10] QuakeLogic, "MOTIONMASTER-6 Stewart platform for advanced 6-DOF testing across industries," *QuakeLogic Blog*. Accessed: Sep. 22, 2026. [Online]. Available: https://blog.quakelogic.net/motionmaster-6-stewart-platform-for-advanced-6-dof-testing-across-industries/

[11] Speedgoat, "Real-time simulation and testing." Accessed: Sep. 22, 2026. [Online]. Available: https://www.speedgoat.com/

[12] Trung Hai IoT, "Xi lanh điện động cơ bước LA03-ST [Stepper motor electric cylinder LA03-ST]." Accessed: Sep. 22, 2026. [Online]. Available: https://trunghaiiot.com/xi-lanh-dien-dong-co-buoc-la03-st/

[13] STMicroelectronics, "NUCLEO-H753ZI." Accessed: Sep. 22, 2026. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-h753zi.html

[14] Bhaskar Dasgupta “The Stewart platform manipulator: a review”, *Mechanism and Machine Theory*, 1999, doi: 10.1016/S0094-114X(99)00006-3. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0094114X99000063

[15] Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Cambridge, U.K.: Cambridge University Press, 2017.

[16] Bosch Rexroth AG, "HELIO by HMI Project," ctrlX World partner page. Accessed: Sep. 22, 2026. [Online]. Available: https://apps.boschrexroth.com/microsites/ctrlx-automation/en/ctrlx-world/partner/helio-by-hmi-project-en/

[17] *Safety of Machinery – General Principles for Design – Risk Assessment and Risk Reduction*, ISO Standard 12100:2010, Nov. 2010.

[18] J.-P. Merlet, *Parallel Robots*, 2nd ed. Dordrecht, Netherlands: Springer, 2006.

[19] *Safety of Machinery – Safety-Related Parts of Control Systems – Part 1: General Principles for Design*, ISO Standard 13849-1:2023, Apr. 2023.

[20] *Safety of Machinery – Electrical Equipment of Machines – Part 1: General Requirements*, IEC Standard 60204-1:2016, Oct. 2016.

[21] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, “Digital twin in manufacturing: A categorical literature review and classification,” *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018, doi: 10.1016/j.ifacol.2018.08.474.

8 Appendices

8.1 Bill of materials

Item Name	Part Number	Qty	Unit Price (VND)	Total Price (VND)	Description
MEAN WELL LRS-350-24 (24V 14.6A)	**LRS-350-24**	1	504000	**504000**	Main 350 W - 24V DC supply for motor drivers
MEAN WELL HDR-30-5 (5V 3A)	**HDR-30-5**	1	347000	**347000**	Isolated 5V DC supply for MCUs and logic
Cầu đấu nối logic IO	**HYBT-15A-1**	4	10000	**40000**	Distribute 5V, 24V Power
Cầu đấu chia nguồn AC					
Cầu đấu chia nguồn DC	**HYBT-15A-2**	4	10000	**40000**	Distribute 5V, 24V Power
Răng lược 5P/6P		3	5000	**15000**	Split 5V and 24Vpower
Cầu đấu PE	**TE JB 2.5/35**	12	15000	**180000**	
Đầu cốt	**SV 2-5**	1	53000	**53000**	
STM32 Nucleo-144 (STM32H753ZI)	**NUCLEO-H753ZI**	1	1950000	**1950000**	Master controller for IK and Web Ethernet
Leadshine DM542-05 Stepper Driver	**DM542-05**	3	830000	**2490000**	Stepper motor driver for linear actuators
Cảm biến quang E3JK-DS100M1	**E3JK-DS100M1 DC12-24V**	1	190000	**190000**	Z direction sensor
DFRobot Gravity 10-DOF IMU (BNO055)	**SEN0253**	1	783000	**783000**	Top-deck absolute platform orientation
CAT6 RJ45 M-F Panel Mount Cable 30cm	**CAT6-RJ45-MF-30**	1	29000	**29000**	External cabinet RJ45 bulkhead port
GX16 4-Pin Aviation Connectors	**GX16-4P**	8	12000	**96000**	Actuator motor power disconnects
GX16 5-Pin Aviation Connector	**GX16-5P**	2	14000	**28000**	IMU sensor cabling disconnect
AC-01 Power Entry (Fuse + Switch)	**AC-01-GRN**	4	20000	**80000**	AC mains inlet with switch and 8A fuse
LAY37-11ZS E-Stop Button (1NO1NC)	**LAY37-11ZS**	2	23000	**46000**	Hardware safety power cut
Aptomat	**NXB-63 2P**	4	87000	**348000**	Stop Electrical For Repair
Start Button		2	22000	**44000**	Start
Stop Button		2	22000	**44000**	Stop
Reset Button		2	22000	**44000**	Reset
16 AWG Power Wire (5m)	**16AWG-5M**	1	90000	**90000**	24V power wiring
22 AWG Logic Wire (10m)	**22AWG-10M**	1	70000	**70000**	5V logic and signal wiring
120 Ohm Termination Resistors	**R-120R-1/4W**	1	10000	**10000**	CAN bus termination
Cabinet Accessories Kit	**CAB-ACC-KIT**	1	250000	**250000**	DIN rail, CABLE Duct, Fan, Screw
Linear Actuator	**LA 03 ST**	3	2800000	**8400000**	3Legs for stewart platform
Trục cacdan / trục ball joint		6	20000	**120000**	
Bạc đạn dưới		6	15000	**90000**	
RC servo	**MG996R**	3	155000	**465000**	Actuator for Working Prototype
Máng nhựa		3	20000	**60000**	
Ốc M2 M2.5 M3 M4 (12, 25, 30, 50 mm)		5	20000		
Cắt CNC Platform + Base		2	50000	**100000**	
				Total Money	**17006000**

8.2 Electrical drawings

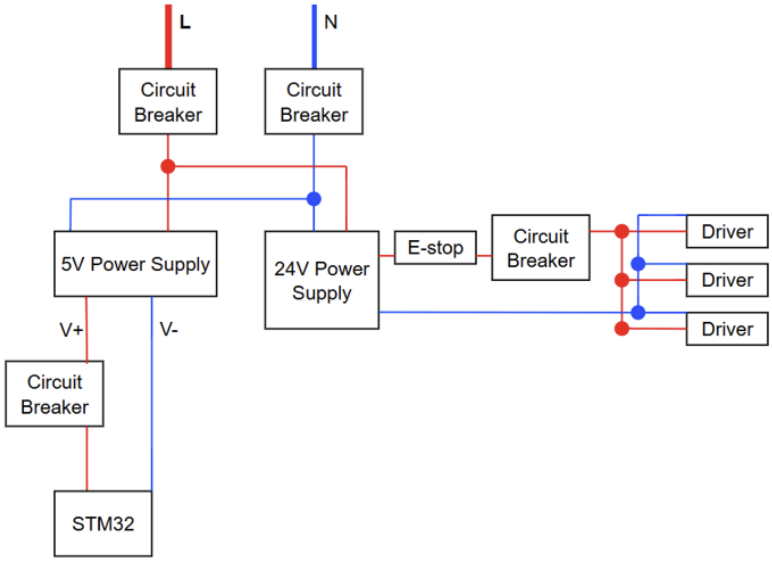
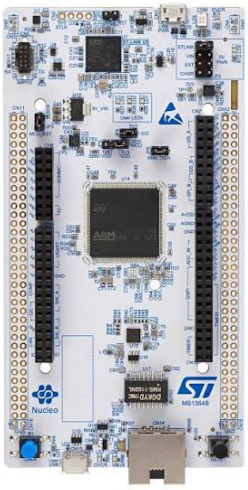


Figure 8-1: Electrical wiring diagram of the control panel

Table 8-1: Control panel components

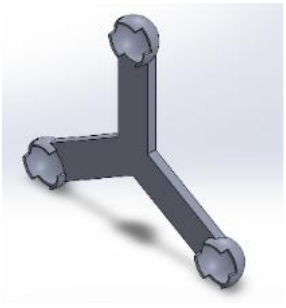
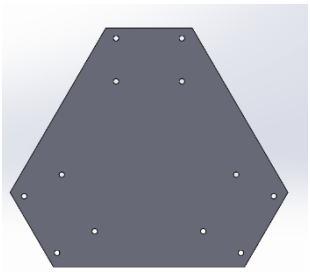
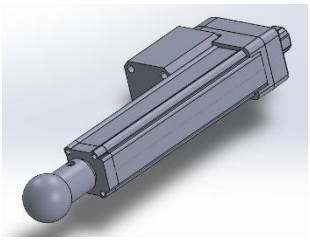
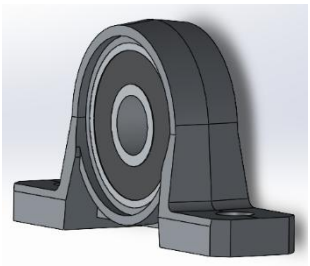
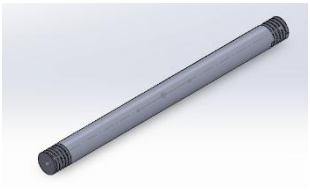
Component name	Description	Figure
Mean Well LRS-350-24	350 W, 24 V DC, 14.6 A switching power supply. Provides power to the three stepper motor drivers.	
MEAN WELL HDR-30-5 (5V 3A)	5 V DC, 3 A power supply. Provides regulated power for the STM32 controller and logic circuits.	
CHINT NXB-63 2P C16 circuit breaker (aptomat)	Two-pole, 16 A circuit breaker. Provides circuit protection and allows the electrical supply to be isolated.	

Component name	Description	Figure
STM32 NUCLEO-H753ZI	Main embedded controller. Processes user commands and IMU readings, performs inverse kinematics and homing, and generates STEP/DIR signals.	
DM542-05 stepper drivers	Microstepping drivers that convert the STEP/DIR/ENA signals from the STM32 into phase currents for each stepper motor. [Add microstep and current settings.]	
DFRobot BNO055 IMU	Measures roll and pitch through onboard sensor fusion and communicates with the STM32 using I²C. Provides an alternative user input for platform movement.	
START and STOP pushbuttons	Connected to STM32 GPIO inputs. START enables the RUNNING state, while STOP requests termination of actuator movement and moves the system to STOPPED.	
Hardwired E-stop and safety relay	Normally closed contact in the relay path that cuts the 24 V supply to the stepper drivers, independent of the STM32 (Stop Category 0).	

8.3 Mechanical drawings

The mechanical components of the industrial Stewart platform are listed in Table 8-2.

Table 8-2: Mechanical components of the industrial Stewart platform

Component name	Description	Figure
Top platform	The distance from the centre of the platform to the centre of each spherical joint is 100 mm.	
Cover	This part is placed and secured on the top platform to add room for placing objects.	
Base	A laser-cut base with holes for mounting the revolute joints.	
Linear actuator	Linear actuator with a ball-joint link at the tip. The distance from the revolute pin to the centre of the ball joint is 233.21 mm when fully retracted, and the travel distance is 200 mm (software-limited to 150 mm).	
Pillow bearing KP08	Serves as the revolute joint at the base, assembled with bolts and fasteners through the holes in the base. Each leg needs 2.	
Pin	A through pin passes through the linear actuator and the pillow bearings to secure the linear actuator. The pin is threaded at both ends for fasteners that restrain its translation along the KP08 rotational axis.	

2D Mechanical Drawings:

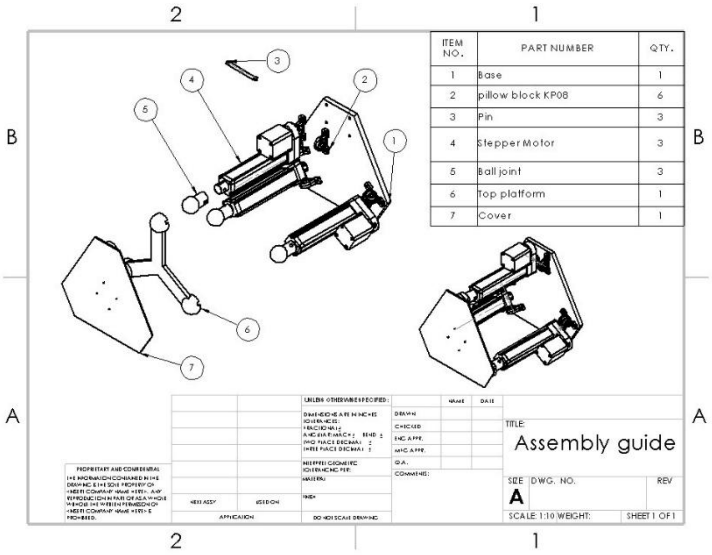


Figure 8-2: Stewart Platform Isometric view

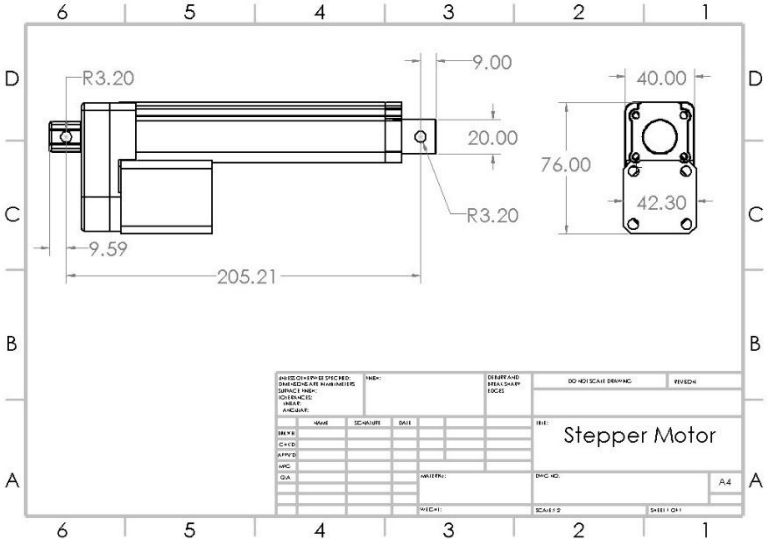


Figure 8-3: Electrical cylinder

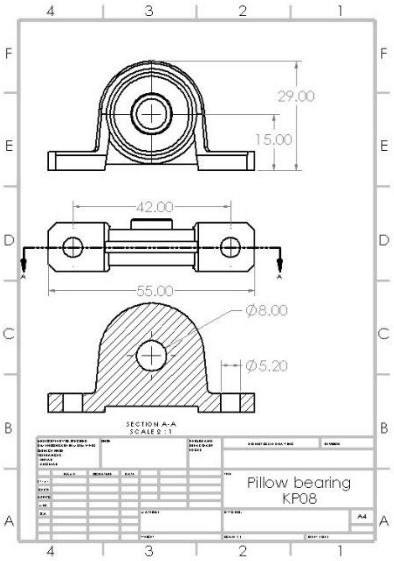


Figure 8-4: KP08 Pillow Bearing

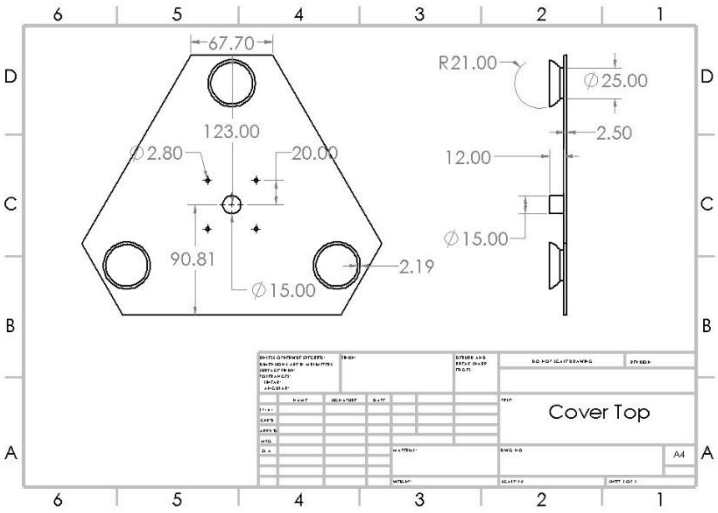


Figure 8-5: 3D printed Cover Top

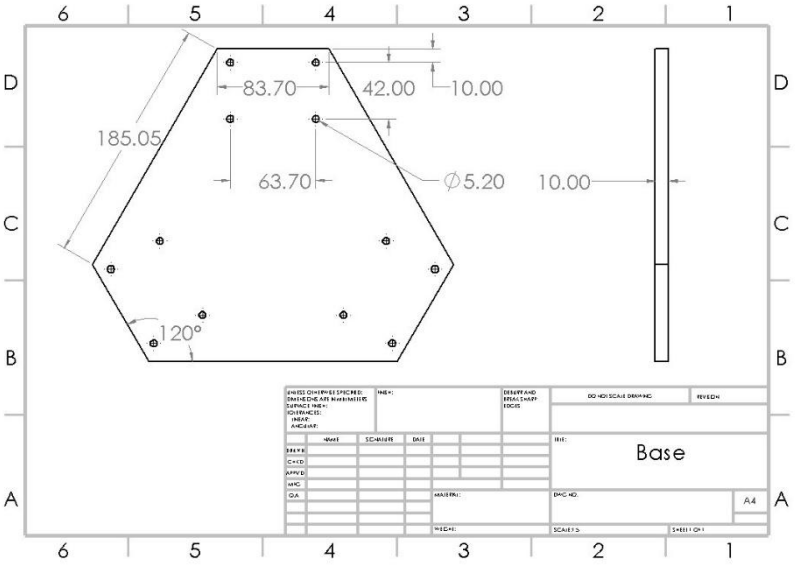


Figure 8-6: Base

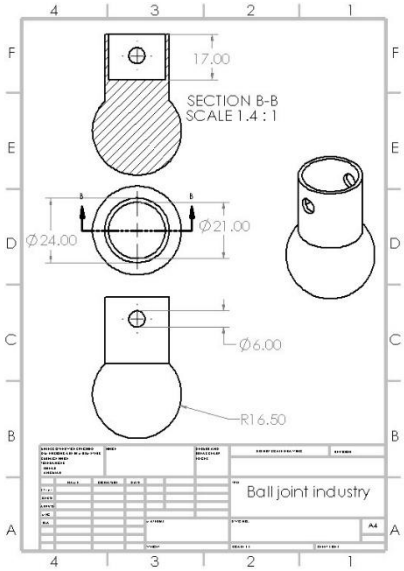


Figure 8-7: 3D printed ball joint

1.

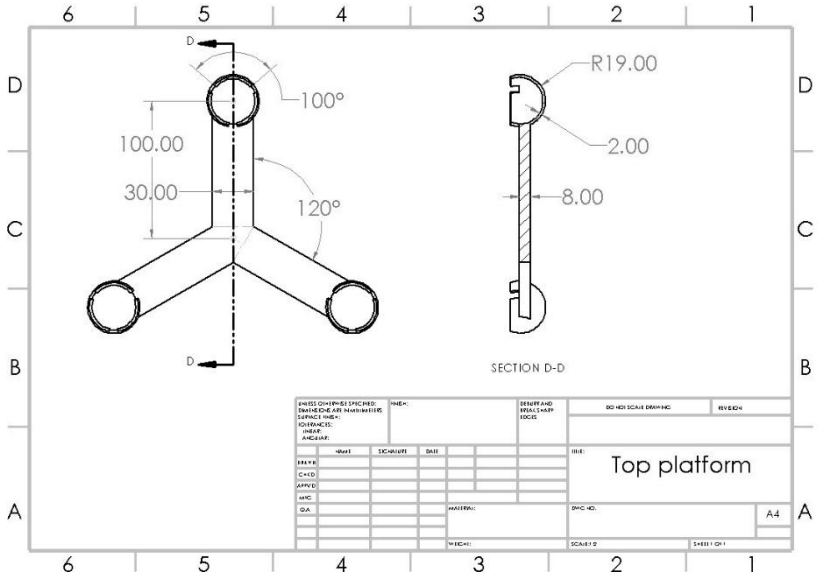


Figure 8-8: 3D printed Top attachment

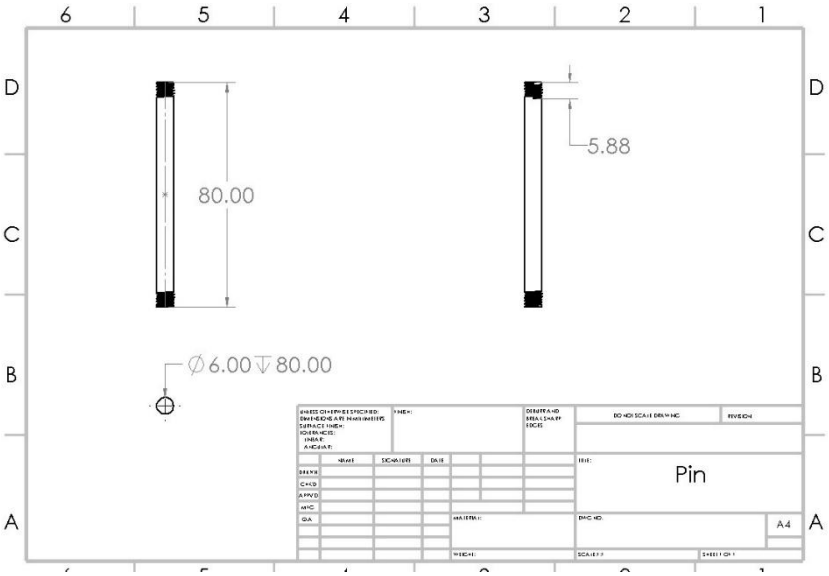


Figure 8-9: Revolute joint pin

8.4 Control code

All control codes can be accessed via github:

https://github.com/LeHuuToan542/OENG1185_EngineeringCapstoneB_TheBoys_3DoFStewartBased

8.5 Project timeline

Project Start Date: 30-Jun-26

Project Name: Capstone Project

Area	Activity	Lead	Supp	Start	End Befc	Days (wo S	Statu	% Do
1	Project Management & Systems Architecture							
	1. System requirement and system structure definement	Dung		1-7-2026	15-7-2026	13	In Progress	75%
	a. Define data structure (Connection)							
	b. Device Communication Protocol							
	2. Work flow and task assign to members	Dung		1-7-2026	15-7-2026	13	Complete	100%
2	Control Algorithm and C++ Generation							
	1. MATLAB/ Simulink Kinematic and Control Design	Dung		10-7-2026	25-7-2026	14	Complete	100%
	a. Understand platform mechanicsl (Orientation-> Leg)							
	b. Develop digital twin model in Matlab							
	c. Design and tune cascaded, closed-loop PID controller							
	d. Verify control algorithm through model behaviour							
	2. Configure and embed Simulink Coder for C++generation.	Dung		25-7-2026	5-8-2026	10	In Progress	75%
	3. Validate and finetune C++ using SIL, PIL and HIL	Dung	Quan	29-7-2026	15-8-2026	16	In Progress	
3	ctrlX CORE PLC Programming							
	1. Understand the controller through the manual	Quan		30-Jun-26	18-Jul-26	17	Complete	100%
	2. Inverse Kinematics Implementation	Quan	Dung	22-Jul-26	22-Aug-26	28	In Progress	25%
	3. Motion Control Verification	Quan		5-Aug-26	22-Aug-26	10		
	4. Debug and Improve	Quan		2-Sep-26	20-Sep-26	16		
4	HMI & Dashboard Development (App Team)							
	1. Bosch HMI Design (CtrlX HMI)	Khang						
	a. Design the visual layout for monitoring position and error alarm			8-7-2026	30-7-2026	20	In Progress	75%
	b. Establish communication link between App to PLC			1-8-2026	10-8-2026	8		
	c. Debugging communication between HMI and other components			10-8-2026	30-8-2026	18		
	2. Integrate IMU data stream into the dashboard app (Optional)	Toan						
5	HIL Implementation							
	1. Configure the plant model to run in 3D viewer.	Toan	Quan	8-7-2026	25-7-2026	16	In Progress	75%
	a. Build the digital twin in 3D viewer							
	b. Validate speed, force, and position compare to simulation							
	c .Simulate external factor (friction, noise, vibration)							
	2. Establish real-time comm between the PLC and the HIL target.	Quan	Toan	1-8-2026	29-8-2026	25		
5	Hardware Integration (Mechanical/Electrical Team)							
	1. Mechanical Setup(Cylinder)	Nguyen	Khang	12-8-2026	29-8-2026	16		
	2. Control Panel and PLC Setup	Quan	Dung	20-8-2026	16-9-2026	24		
	a. Control panel layout							
	b. PLC I/O to Device							
	3. Electrical & Drive Integration	Toan	Quan	20-8-2026	16-9-2026	24		
	a. Wire the 3 actuators, sensors, and PS to the PLC drive modules.							
	b. Tune physical actuator drives and verify sensor calibration.							

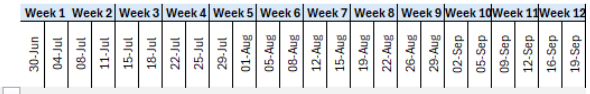


Figure 8-10: Project Gantt chart (Completion Plan schedule)

9 Meeting journals

The Boys
Weekly meeting
Meeting No:1

Meeting Details

Date:	01-July-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	What are some breakthroughs over the semester break?
2	What is the completion plan for the whole semester?

Action Items

No	Item	Who	By
1	The project completion plan should be divided into many detailed segments	All members	19-July-2026
2	Validate Control Algorithm in Simulink	Dung & Toan	7-July-2026
3	Prepare hardware lists	All members	20-July-2026

RMIT UNIVERSITY

The Boys
Weekly meeting
Meeting No:2

Meeting Details

Date:	08-July-2026
Time:	15:30-16:30
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	What are the details of the completion plan?
2	What hardware is needed? Why does our team need them?
3	The system architecture of the whole system

Action Items

No	Item	Who	By
1	Finish the finalized version of the completion plan to submit as assignment A1	All members	17-July-2026
2	Justification for all the needed hardware	All members	17-July-2026
3	Exploring and practicing Bosch manuals	Khang, Quan & Nguyen	27-July-2026
4	Finalize Control Algorithm	Dung & Toan	27-July-2026

RMIT UNIVERSITY

The Boys
Weekly meeting
Meeting No: 3

Meeting Details

Date:	15-July-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review all the hardware needed to purchase
2	Specify the application of the Stewart Platform
3	Perform more literature review to deeply understand the Stewart Kinematics
4	Prepare individual completion plans

Action Items

No	Item	Who	By
1	Provide a detailed individual working plan for the semester	All members	22-July-2026
2	Finalize the hardware list	All members	17-July-2026
3	Initiate ideas and designs for a working prototype	Quan, Khang & Nguyen	23-July-2026
4	Virtual ctrlX CORE PLC Programming	Quan, Khang & Nguyen	23-August-2026
5	Initiating idea for industrial system with physical linear actuators	Dung & Toan	23-July-2026

The Boys
Weekly meeting
Meeting No: 4

Meeting Details

Date:	22-July-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Proposal for working prototype & industrial system
2	Issue with Bosch ctrlX CORE
3	Find ways to collect and validate data

Action Items

No	Item	Who	By
1	Find alternative ways to compensate for the missing physical ctrlX CORE Controller	All member	25-July-2026
2	Finish the working prototype to verify the inverse kinematics	Quan, Khang & Nguyen	30-July-2026
3	Purchase STM32 to replace physical ctrlX CORE to control 3 linear industrial actuators	Dung & Toan	05-August-2026

The Boys
Weekly meeting
Meeting No: 5

Meeting Details

Date:	29-July-2026
Time:	15:15-16:15
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review the progress of the Stewart Platform inverse kinematics and verify the calculated actuator lengths.
2	Discuss the design and construction of the physical working prototype.
3	Discuss the dimensions of the base platform, top platform and actuator mechanism.
4	Review the progress of the Arduino-based prototype and its rack-and-pinion mechanism.

Action Items

No	Item	Who	By
1	Finalize the prototype mechanical dimensions	All members	2-Aug-2026
2	Complete Arduino control for the prototype	Dung & Toan	2-Aug-2026
3	Test inverse kinematics using different platform poses	All members	4-Aug-2026

The Boys
Weekly meeting
Meeting No: 6

Meeting Details

Date:	05-August-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review the working prototype and inverse kinematics testing results.
2	Discuss the industrial Stewart Platform dimensions and actuator requirements.
3	Discuss the Bosch ctrlX CORE implementation and digital twin development.
4	Review the possibility of using Bosch hardware for Hardware-in-the-Loop testing.

Action Items

No	Item	Who	By
1	Finalize industrial Stewart Platform dimensions	Quan & Toan	9-Aug-2026
2	Continue development of the Bosch digital twin	Dung & Toan	12-Aug-2026
3	Investigate controller and HIL implementation options	Dung & Nguyen	12-Aug-2026

RMIT UNIVERSITY

The Boys
Weekly meeting
Meeting No: 7

Meeting Details

Date:	12-August-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review the Bosch digital twin and 3D Viewer progress.
2	Discuss the implementation of inverse kinematics in the controller.
3	Discuss the availability of Bosch ctrlX CORE hardware.
4	Plan an alternative controller solution if the Bosch hardware cannot be provided.

Action Items

No	Item	Who	By
1	Complete the Stewart Platform model in the Bosch 3D Viewer	Khang & Quan	16-Aug-2026
2	Prepare the inverse kinematics implementation for STM32	Dung & Toan	16-Aug-2026
3	Purchase and prepare the STM32 development board	All members	16-Aug-2026
4	Prepare the communication method between STM32 and Simulink	Nguyen & Dung	19-Aug-2026

RMIT UNIVERSITY

The Boys
Weekly meeting
Meeting No: 8

Meeting Details

Date:	19-August-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review STM32 controller development and inverse kinematics implementation.
2	Discuss communication between STM32 and the Simulink digital twin.
3	Review the PIL testing procedure.
4	Discuss the remaining mechanical, electrical and software integration tasks.

Action Items

No	Item	Who	By
1	Complete STM32 inverse kinematics implementation	Dung & Toan	23-Aug-2026
2	Establish STM32–Simulink serial communication	Nguyen, Dung & Toan	30-Aug-2026
3	Conduct initial PIL testing	Toan, Khang and Nguyen	02-Sep-2026
4	Continue electrical wiring and control panel assembly	All members	02-Sep-2026

RMIT UNIVERSITY

The Boys
Weekly meeting
Meeting No: 9

Meeting Details

Date:	26-August-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review the progress of STM32 programming and linear actuator control.
2	Discuss the implementation of stepper motor control, limit switches and homing functions.
3	Review the progress of the Bosch ctrlX CORE PLC and 3D Viewer development.
4	Discuss communication methods and remaining integration challenges between STM32 and ctrlX CORE.

Action Items

No	Item	Who	By
1	Continue STM32 programming and test linear actuator movement.	Dung & Toan	30-Aug-2026
2	Complete the PLC control and 3D Viewer implementation.	Khang, Quan & Nguyen	30-Aug-2026
3	Conduct initial PIL testing	Toan, Nguyen, Khang	02-Sep-2026
4	Continue electrical wiring and control panel assembly.	All members	02-Sep-2026

The Boys
Weekly meeting
Meeting No: 10

Meeting Details

Date:	02-September-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review the progress of the physical linear actuator and STM32 control system.
2	Discuss the implementation and testing of the homing function and actuator safety limits.
3	Review PLC control of the Stewart Platform model in the Bosch 3D Viewer.
4	Discuss the remaining communication and integration issues between the physical and virtual systems.

Action Items

No	Item	Who	By
1	Complete STM32 motor control, homing and limit switch testing.	Dung & Toan	06-Sep-2026
2	Test the PLC program and 3D Viewer movement using different input values	Khang, Quan & Nguyen	06-Sep-2026
3	Finalize the electrical connections and control panel assembly	All members	09-Sep-2026
4	Begin preparing the final report and presentation materials	All members	09-Sep-2026

RMIT UNIVERSITY

The Boys
Weekly meeting
Meeting No: 11

Meeting Details

Date:	09-September-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review the final testing results of the STM32-controlled linear actuator.
2	Confirm successful PLC control of the Stewart Platform model in the Bosch 3D Viewer.
3	Discuss the project limitations, as the physical and virtual control systems are currently operating independently.
4	Review the final report progress and plan the demonstration procedure for the campus exhibition.

Action Items

No	Item	Who	By
1	Finalize the STM32 control program and prepare the physical actuator demonstration.	Dung & Toan	13-Sep-2026
2	Finalize the virtual ctrlX CORE and 3D Viewer demonstration.	Khang, Quan & Nguyen	13-Sep-2026
3	Complete the technical sections of the final report, including results and limitations.	All members	15-Sep-2026
4	Prepare the presentation slides and assign presentation responsibilities.	All members	15-Sep-2026

RMIT UNIVERSITY

The Boys
Weekly meeting
Meeting No: 12

Meeting Details

Date:	16-September-2026
Time:	15:00-16:00
Attendees:	Dr. Nguyen Hai Nguyen
	Mr. Nguyen Hoang Thinh, Mr. Nguyen Hung Binh, Mr. Krasilnikov Aleksandr
	Pham Tri Dung, Le Huu Toan, Wong Vi Khang, Vu Tran Khoi Nguyen, Pham Quan (joined via Google Meet)
Apologies:	
Copy To:	

Information /Decision

Item No.	Discussion Summary
1	Review the final project achievements: successful virtual ctrlX CORE control of the 3D Viewer and STM32 control of the physical linear actuator.
2	Confirm the final project scope and limitations, including the lack of completed communication between the physical and virtual systems.
3	Review the final report and presentation slides and organize the presentation rehearsal.
4	Plan the pre-recorded presentation video and final preparations for the campus demonstration and exhibition.

Action Items

No	Item	Who	By
1	Finalize and review the project report and presentation slides.	All members	20-Sep-2026
2	Rehearse the presentation and prepare individual speaking scripts.	All members	20-Sep-2026
3	Record and edit the pre-recorded presentation video during the weekend.	All members	20-Sep-2026
4	Prepare and test the physical actuator, 3D Viewer and equipment for the campus exhibition.	All members	20-Sep-2026